|

Hands-on Activity 9.1

|

Data Visualization using Pandas and Matplotlib

|
|--- | --- |
Name: | Rubang, Jethro Aaron S.

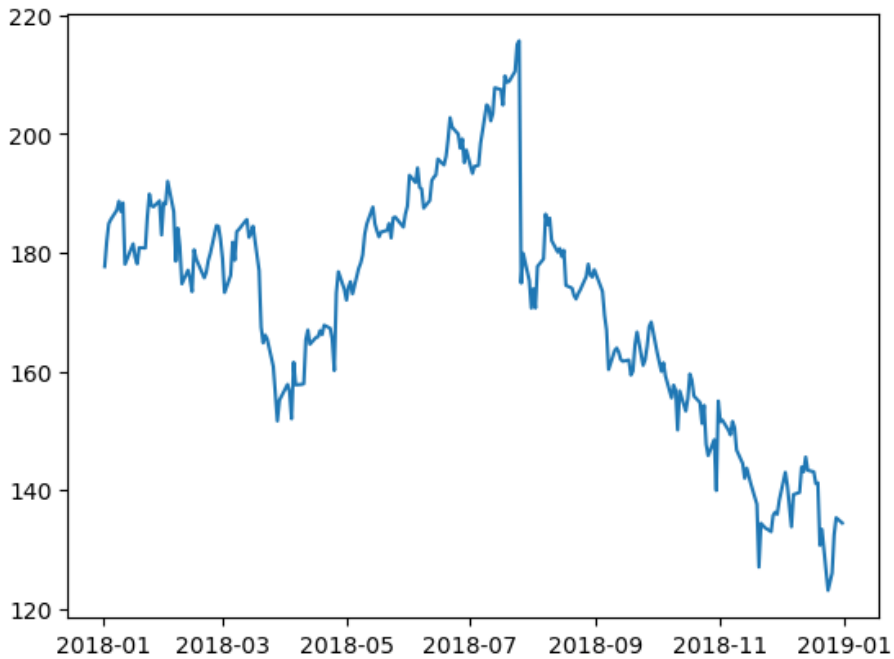
Course and Section: | CPE 311-CPE22S3

Instructor: | Eng. Neal Barton James Matira
Date Performed: | March 3, 2026
Date Submitted: | March 3, 2026

Introduction to Matplotlib

In [1]:
```
import matplotlib.pyplot as plt
import pandas as pd
```

In [2]:
```
fb = pd.read_csv('fb_stock_prices_2018.csv', index_col='date',
parse_dates=True)

plt.plot(fb.index, fb.open)
plt.show()
```



The first line plot of the open price of Facebook indicates the daily fluctuations of the price in 2018. The trend shows the fluctuations of the stock market as to when growth is experienced and when it declines.

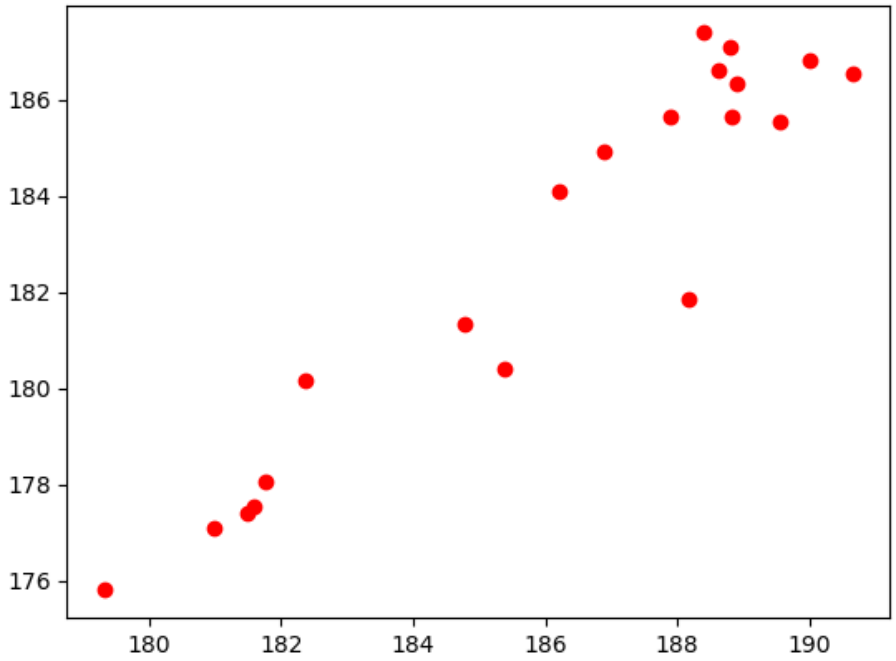
In [4]:
```
import matplotlib.pyplot as plt
import pandas as pd
fb = pd.read_csv(
'fb_stock_prices_2018.csv', index_col='date', parse_dates=True
)
plt.plot(fb.index, fb.open)
```

Out[4]: [<matplotlib.lines.Line2D at 0x198cf9df860>]

The plot of the high vs low with the combination of red circles visualizes the relationship between the highest and the lowest price per day. The relationship between the high and low prices is strong positive relationship as the points move in the same direction.

In [5]:
```
plt.plot('high', 'low', 'ro', data=fb.head(20))
```

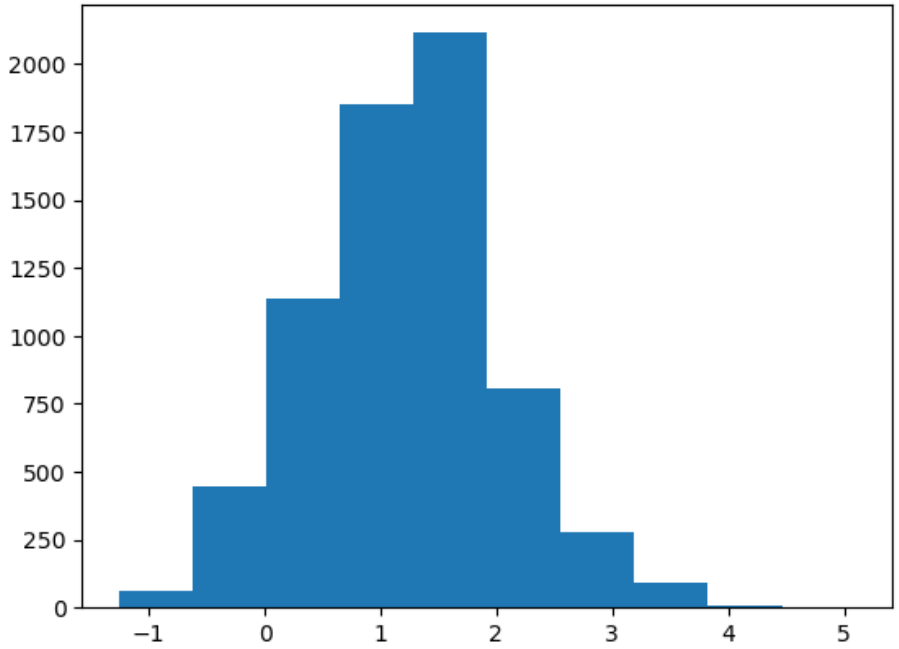
Out[5]:
```
[<matplotlib.lines.Line2D at 0x198cdff1580>]
```



The histogram of the magnitude of earthquakes (magtype = ml) indicates that the greatest earthquakes are in the lower side. Adding additional bins would give more specific patterns in distribution, but would add noise.

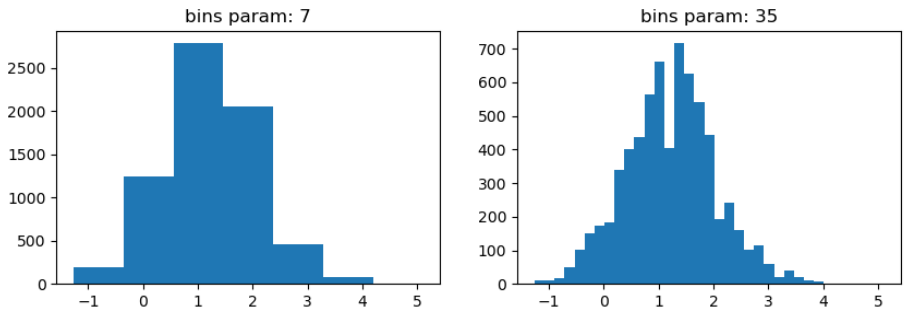
In [8]:
```
quakes = pd.read_csv('earthquakes.csv')
plt.hist(quakes.query('magType == "ml"').mag)
```

Out[8]:
```
(array([6.400e+01, 4.450e+02, 1.137e+03, 1.853e+03, 2.114e+03,
8.070e+02,
        2.800e+02, 9.200e+01, 9.000e+00, 2.000e+00]),
 array([-1.26 , -0.624,  0.012,  0.648,  1.284,  1.92 ,  2.556,
3.192,
         3.828,  4.464,  5.1  ]),
 <BarContainer object of 10 artists>)
```



Subplots and grid specifications illustrate the arrangement of several visualizations in a single figure. This enhances the comparison of datasets.

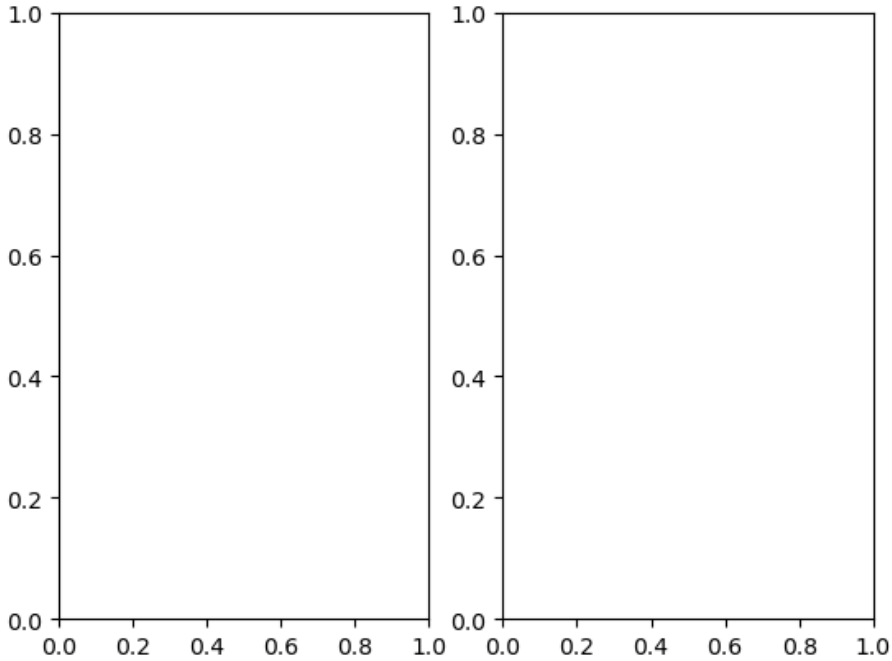
In [10]:
```
x = quakes.query('magType == "ml"').mag
fig, axes = plt.subplots(1, 2, figsize=(10, 3))
for ax, bins in zip(axes, [7, 35]):
    ax.hist(x, bins=bins)
    ax.set_title(f'bins param: {bins}')
```



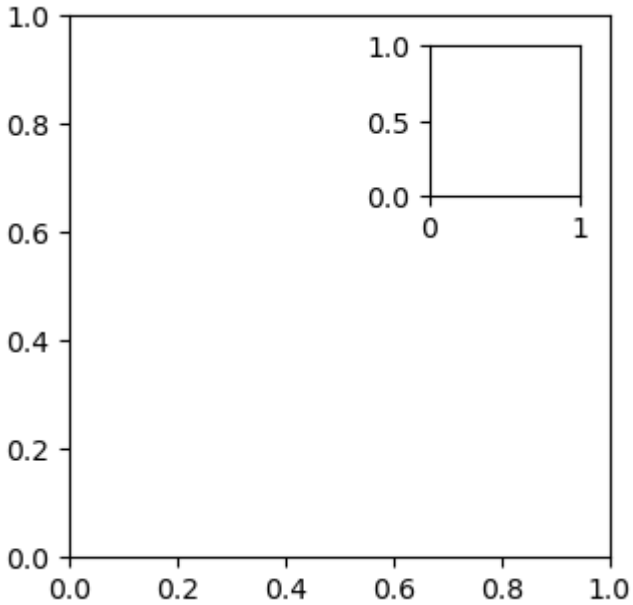
In [11]:
```
fig = plt.figure()
```

```
<Figure size 640x480 with 0 Axes>
```

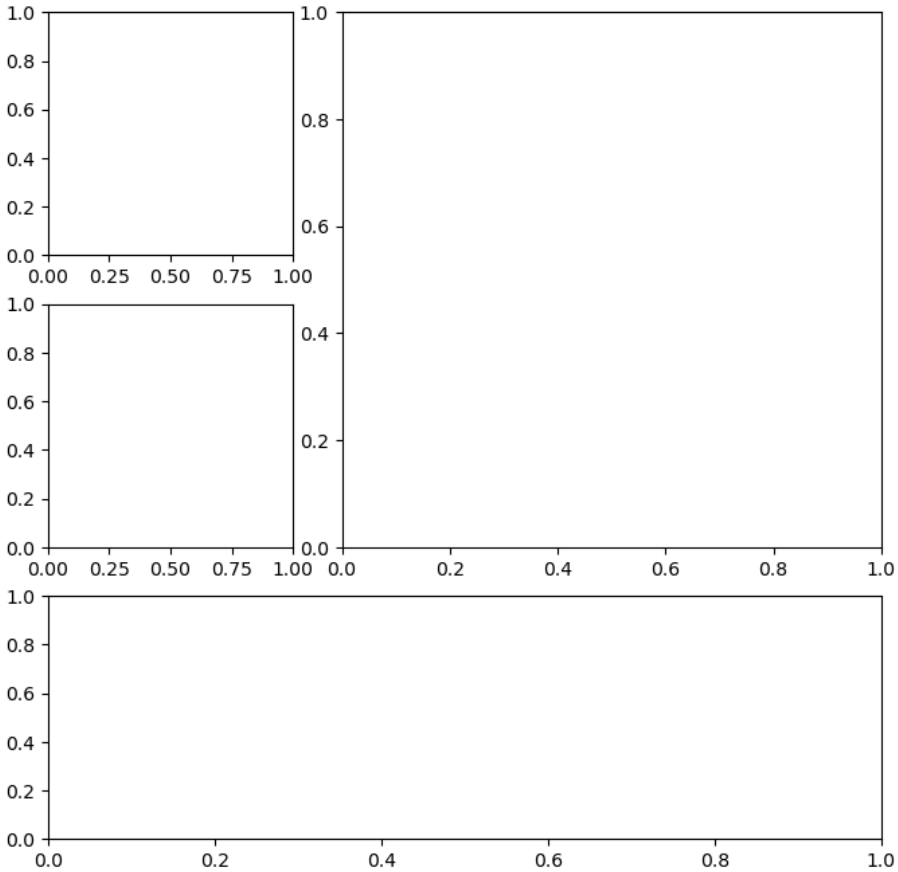
In [12]:
```
fig, axes = plt.subplots(1, 2)
```



In [13]:
```
fig = plt.figure(figsize=(3, 3))
outside = fig.add_axes([0.1, 0.1, 0.9, 0.9])
inside = fig.add_axes([0.7, 0.7, 0.25, 0.25])
```



In [14]:
```
fig = plt.figure(figsize=(8, 8))
gs = fig.add_gridspec(3, 3)
top_left = fig.add_subplot(gs[0, 0])
mid_left = fig.add_subplot(gs[1, 0])
top_right = fig.add_subplot(gs[:2, 1:])
bottom = fig.add_subplot(gs[2,:])
```



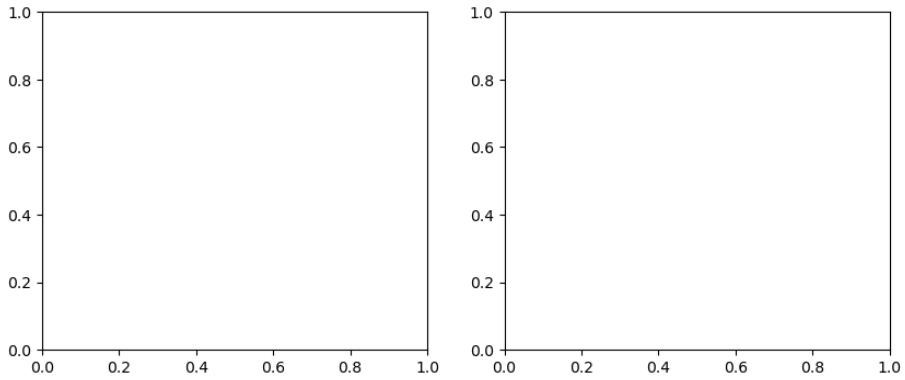
In [15]:
```
fig.savefig('empty.png')
```

In [16]:
```
plt.close('all')
```

In [17]:
```
fig = plt.figure(figsize=(10, 4))
```

```
<Figure size 1000x400 with 0 Axes>
```

In [18]:
```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
```



On the whole, this part illustrates the simplest plotting, management of figures, layout customization, and configuration control in Matplotlib.

In [19]:
```
import random
import matplotlib as mpl
rcparams_list = list(mpl.rcParams.keys())
random.seed(20) # make this repeatable
random.shuffle(rcparams_list)
sorted(rcparams_list[:20])
```

Out[19]:
```
['axes.edgecolor',
 'axes.formatter.use_locale',
 'boxplot.flierprops.markeredgewidth',
 'boxplot.showfliers',
 'contour.negative_linestyle',
 'date.autoformatter.month',
 'date.epoch',
 'figure.autolayout',
 'image.resample',
 'keymap.grid_minor',
 'legend.facecolor',
 'legend.labelcolor',
 'patch.facecolor',
 'text.parse_math',
 'webagg.open_in_browser',
 'xaxis.labellocation',
 'xtick.alignment',
 'xtick.labelcolor',
 'yaxis.labellocation',
 'ytick.minor.pad']
```

In [20]:
```
mpl.rcParams['figure.figsize']
```

Out[20]:
```
[6.4, 4.8]
```

In [21]:
```
mpl.rcParams['figure.figsize'] = (300, 10)
mpl.rcParams['figure.figsize']
```

Out[21]: [300.0, 10.0]

In [22]:
```
mpl.rcdefaults()
mpl.rcParams['figure.figsize']
```

Out[22]: [6.4, 4.8]

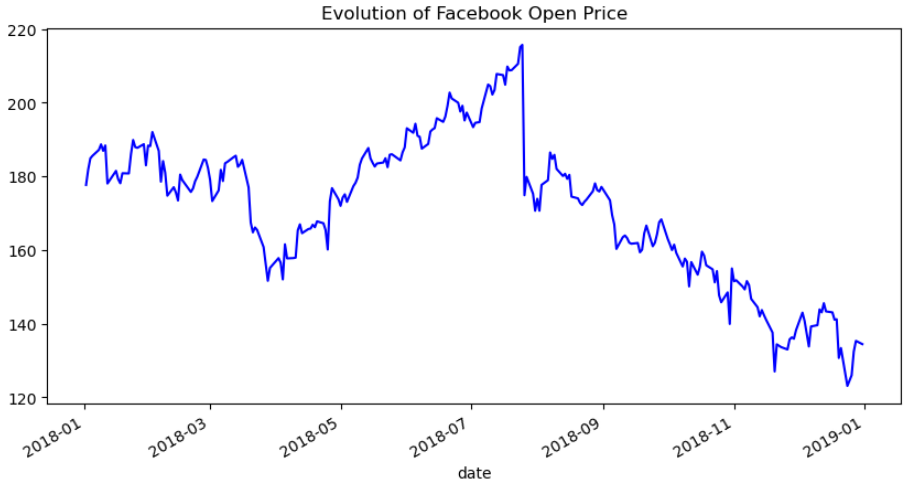
In [23]:
```
plt.rc('figure', figsize=(20, 20)) # change figsize default to (20,
20)
plt.rcdefaults() # reset the default
```

Plotting with Pandas

In [24]:
```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
fb = pd.read_csv('fb_stock_prices_2018.csv', index_col='date',
parse_dates=True
)
quakes = pd.read_csv('earthquakes.csv')
```

In [25]:
```
fb.plot(
    kind='line',
    y='open',
    figsize=(10, 5),
    style='b-',
    legend=False,
    title='Evolution of Facebook Open Price'
)
```

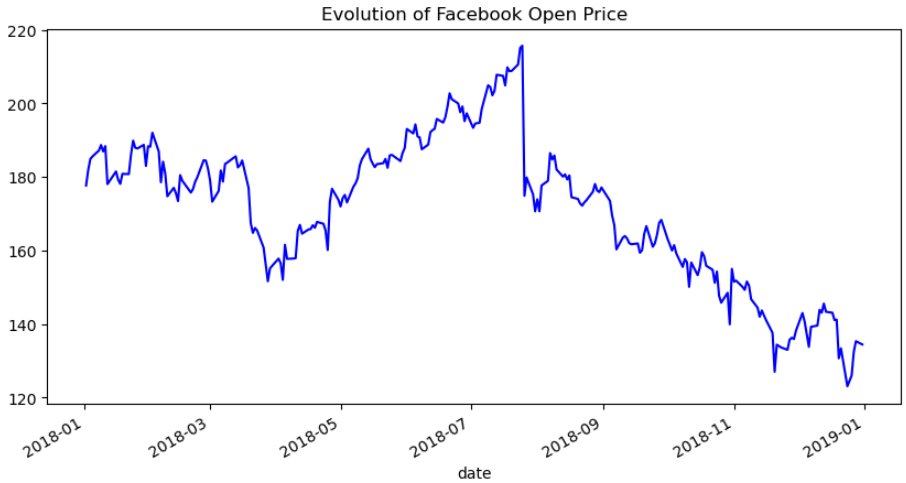
Out[25]:
```
<Axes: title={'center': 'Evolution of Facebook Open Price'},
xlabel='date'>
```



Facebook open price line plots prove the overall oscillating trend during 2018, including the definite peaks and declines.

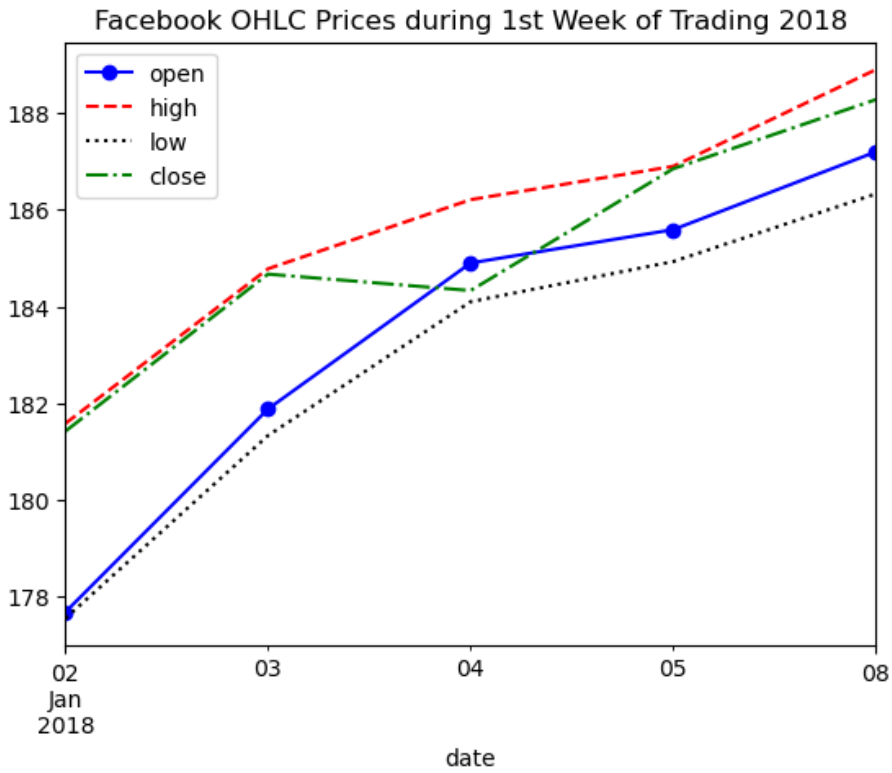
In [26]:
```
fb.plot(
    kind='line',
    y='open',
    figsize=(10, 5),
    color='blue',
    linestyle='solid',
    legend=False,
    title='Evolution of Facebook Open Price'
)
```

Out[26]:
```
<Axes: title={'center': 'Evolution of Facebook Open Price'},
xlabel='date'>
```



In [27]:
```
fb.iloc[:5,].plot(
    y=['open', 'high', 'low', 'close'],
    style=['b-o', 'r--', 'k:', 'g-.'],
    title='Facebook OHLC Prices during 1st Week of Trading 2018'
)
```

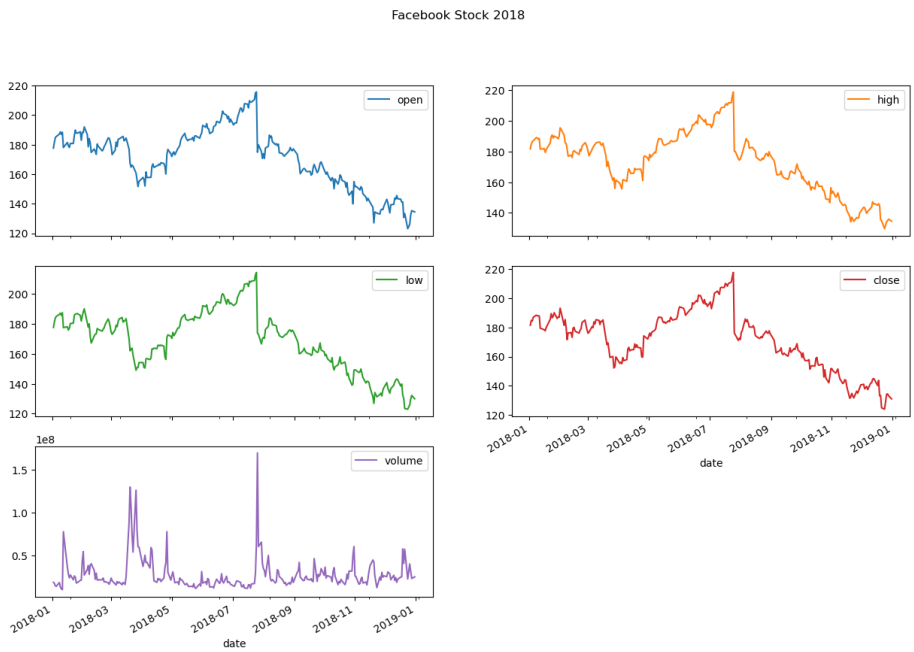
Out[27]:
```
<Axes: title={'center': 'Facebook OHLC Prices during 1st Week
of Trading 2018'}, xlabel='date'>
```



Drawing OHLC price (Open, High, Low and Close) price reveals that all the price indicators move together in the same direction.

In [28]:
```
fb.plot(
    kind='line',
    subplots=True,
    layout=(3,2),
    figsize=(15,10),
    title='Facebook Stock 2018'
)
```

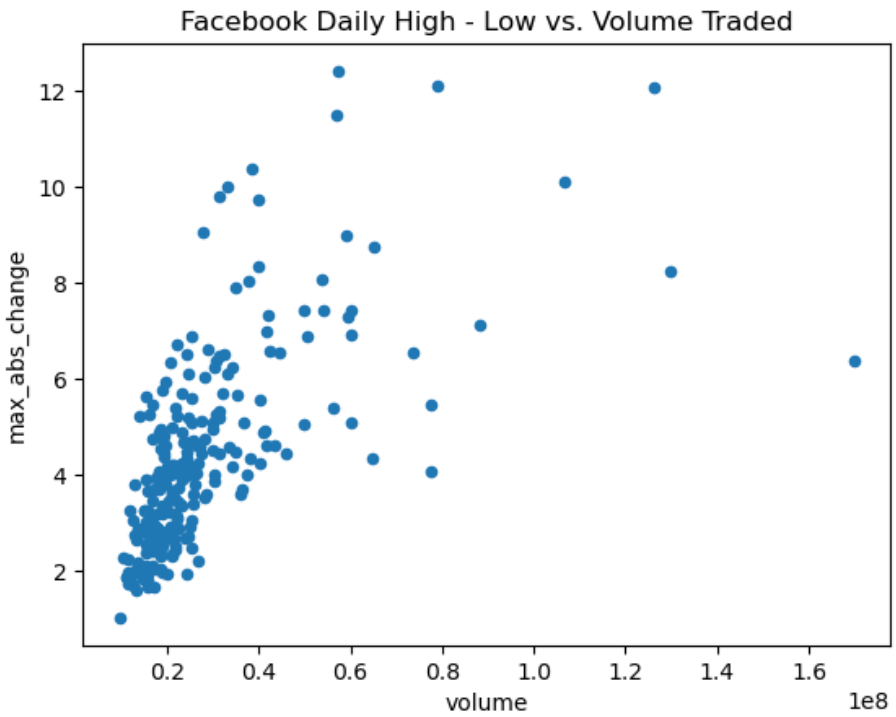
Out[28]:
```
array([[<Axes: xlabel='date'>, <Axes: xlabel='date'>],
       [<Axes: xlabel='date'>, <Axes: xlabel='date'>],
       [<Axes: xlabel='date'>, <Axes: xlabel='date'>]],
      dtype=object)
```



Subplots enable the analysis of the trends of each stock variable separately and more clearly.

In [29]:
```
fb.assign(
    max_abs_change=fb.high - fb.low
).plot(
    kind='scatter', x='volume', y='max_abs_change',
    title='Facebook Daily High - Low vs. Volume Traded'
)
```

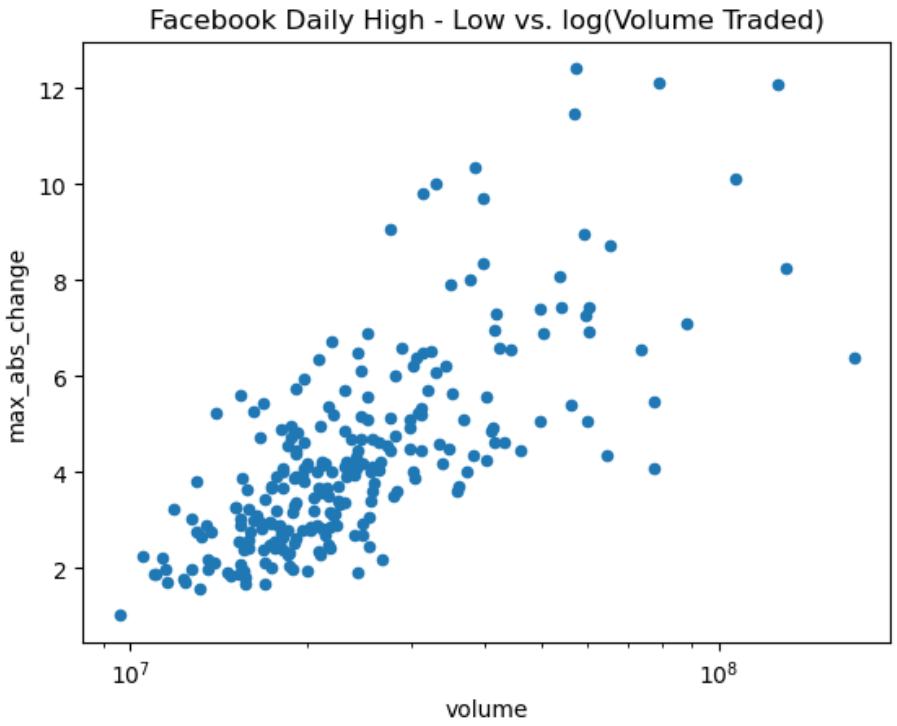
Out[29]:
```
<Axes: title={'center': 'Facebook Daily High - Low vs. Volume
Traded'}, xlabel='volume', ylabel='max_abs_change'>
```



It can be seen that the positive relationship exists between the volume and the max abs change in the scatter plot of volume vs max abs change: the greater the trading volume, the greater the daily price change is likely to be.

In [30]:
```
fb.assign(
    max_abs_change=fb.high - fb.low
).plot(
    kind='scatter', x='volume', y='max_abs_change',
    title='Facebook Daily High - Low vs. log(Volume Traded)',
    logx=True
)
```

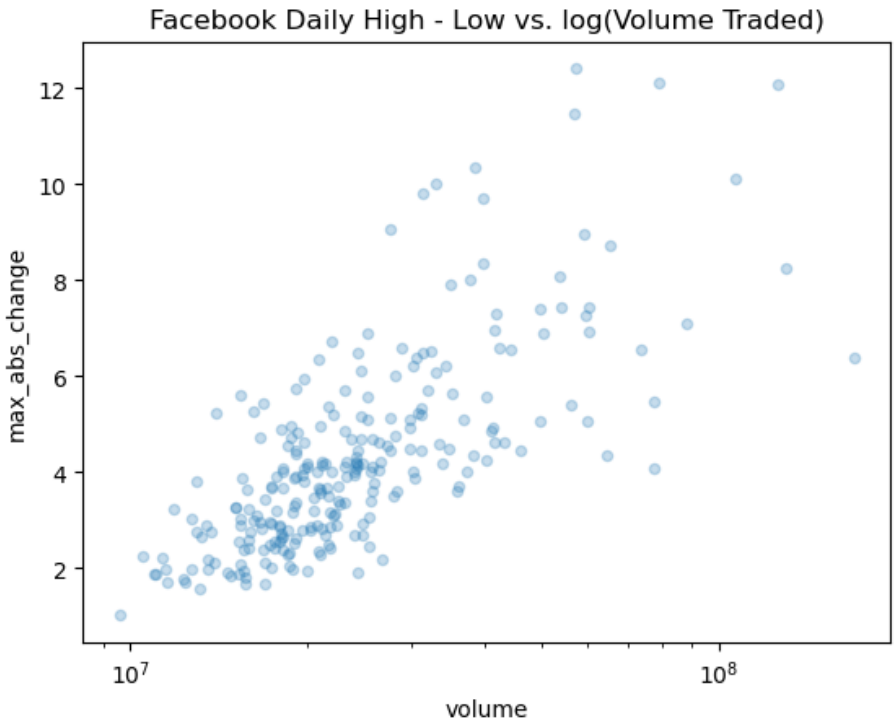
Out[30]:
```
<Axes: title={'center': 'Facebook Daily High - Low vs.
log(Volume Traded)'}, xlabel='volume', ylabel='max_abs_change'>
```



Logarithmic scaling of volume is better as there is a large variation in volumes of stocks.

In [31]:
```
fb.assign(
    max_abs_change=fb.high - fb.low
).plot(
    kind='scatter', x='volume', y='max_abs_change',
    title='Facebook Daily High - Low vs. log(Volume Traded)',
    logx=True, alpha=0.25
)
```

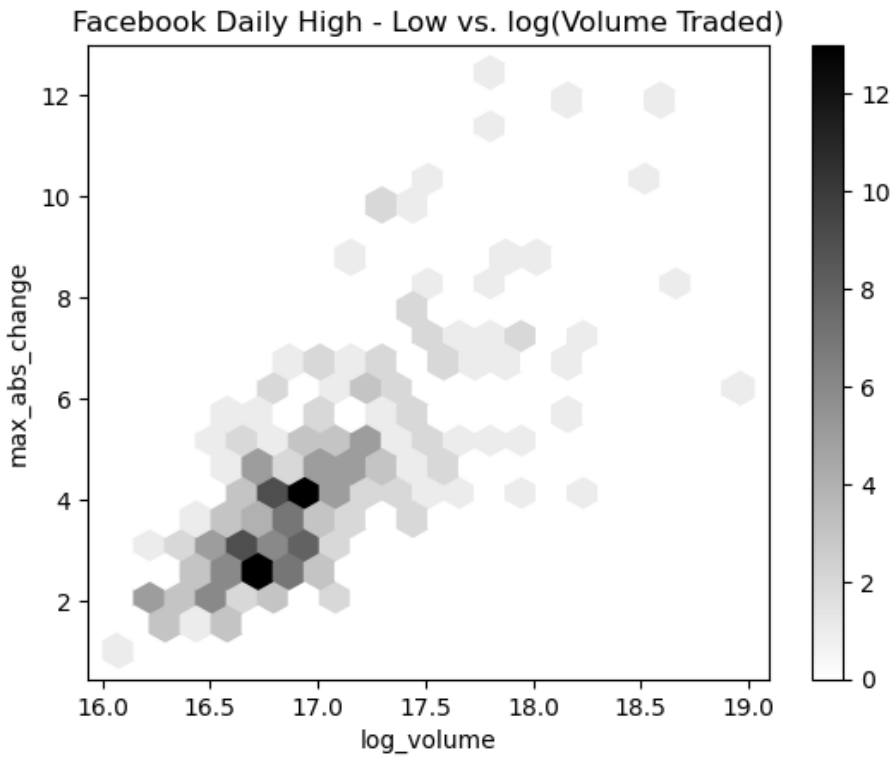
Out[31]:
```
<Axes: title={'center': 'Facebook Daily High - Low vs.
log(Volume Traded)'}, xlabel='volume', ylabel='max_abs_change'>
```



In the correlation matrix heatmap, there are significant positive correlations between the open, high, low and close prices. Volume is not as associated with price variables but more associated with volatility (max_abs_change).

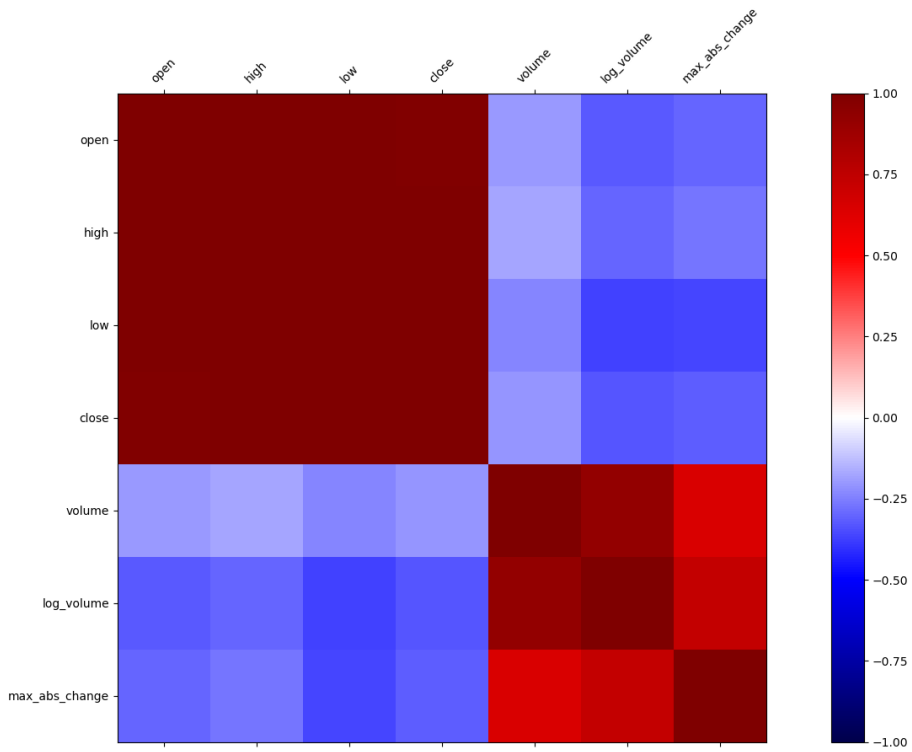
In [32]:
```
fb.assign(
    log_volume=np.log(fb.volume),
    max_abs_change=fb.high - fb.low
).plot(
    kind='hexbin',
    x='log_volume',
    y='max_abs_change',
    title='Facebook Daily High - Low vs. log(Volume Traded)',
    colormap='gray_r',
    gridsize=20,
    sharex=False # we have to pass this to see the x-axis due to a
bug in this version of pandas
)
```

Out[32]:
```
<Axes: title={'center': 'Facebook Daily High - Low vs.
log(Volume Traded)'}, xlabel='log_volume',
ylabel='max_abs_change'>
```



In [34]:
```
fig, ax = plt.subplots(figsize=(20, 10))

fb_corr = fb.assign(
    log_volume=np.log(fb.volume),
    max_abs_change=fb.high - fb.low
).corr()

im = ax.matshow(fb_corr, cmap='seismic')
im.set_clim(-1, 1)   # <-- Fix here

fig.colorbar(im)

labels = [col.lower() for col in fb_corr.columns]
ax.set_xticks(range(len(labels)))
ax.set_yticks(range(len(labels)))
ax.set_xticklabels(labels, rotation=45)
ax.set_yticklabels(labels)
```

Out[34]:
```
[Text(0, 0, 'open'),
 Text(0, 1, 'high'),
 Text(0, 2, 'low'),
 Text(0, 3, 'close'),
 Text(0, 4, 'volume'),
 Text(0, 5, 'log_volume'),
 Text(0, 6, 'max_abs_change')]
```

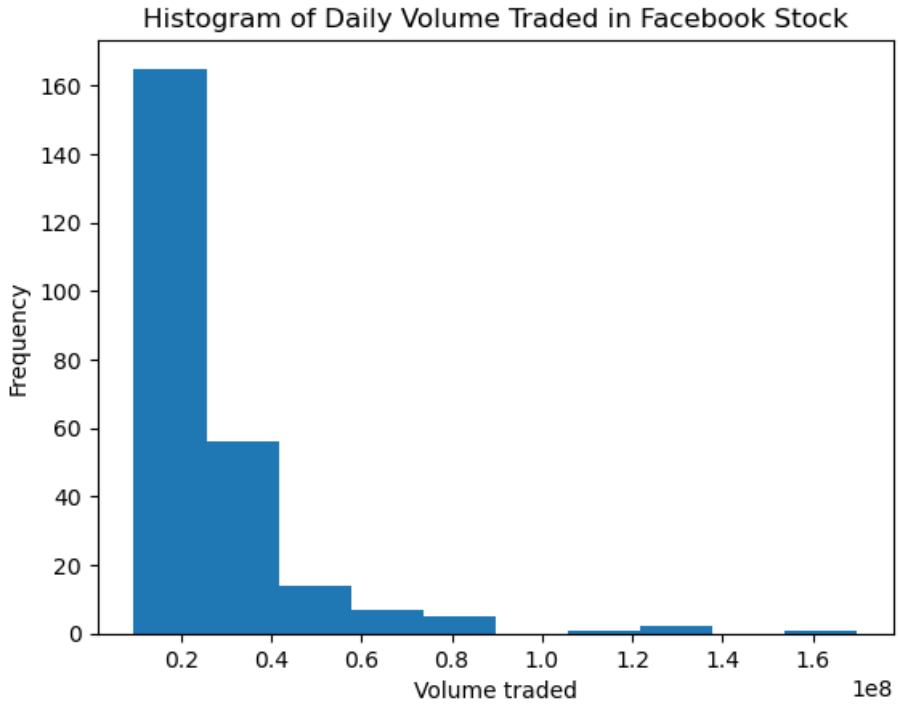


In [35]:
```
fb_corr.loc['max_abs_change', ['volume', 'log_volume']]
```

Out[35]:
```
volume        0.642027
log_volume    0.731542
Name: max_abs_change, dtype: float64
```

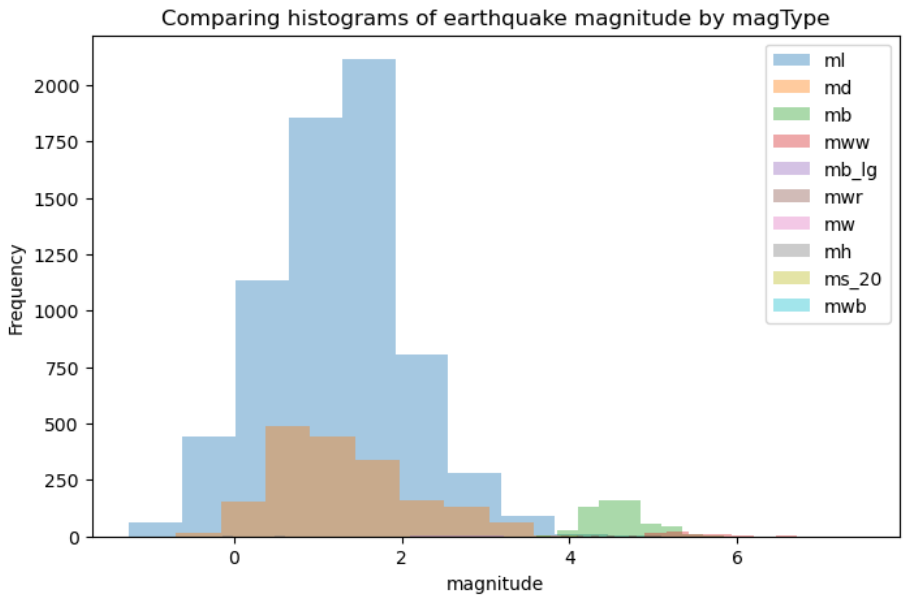
In [36]:
```
fb.volume.plot(
    kind='hist',
    title='Histogram of Daily Volume Traded in Facebook Stock'
)
plt.xlabel('Volume traded') # label the x-axis (discussed in chapter
6)
```

Out[36]: Text(0.5, 0, 'Volume traded')



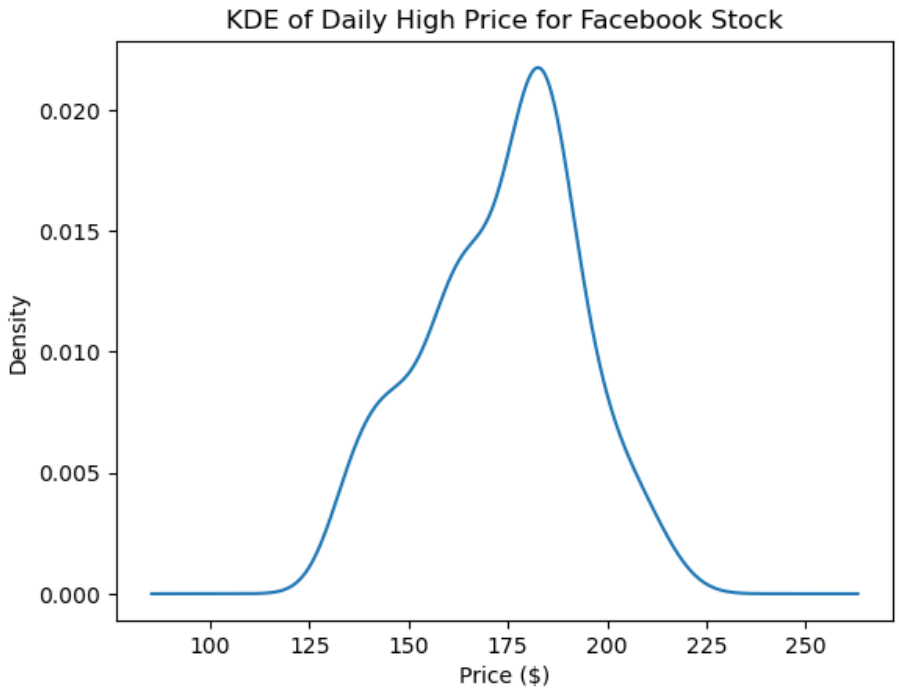
In [37]:
```
fig, axes = plt.subplots(figsize=(8, 5))
for magtype in quakes.magType.unique():
    data = quakes.query(f'magType == "{magtype}"').mag
    if not data.empty:
        data.plot(
            kind='hist', ax=axes, alpha=0.4,
            label=magtype, legend=True,
            title='Comparing histograms of earthquake magnitude by
magType'
)
plt.xlabel('magnitude') # label the x-axis (discussed in chapter 6)
```

Out[37]: Text(0.5, 0, 'magnitude')



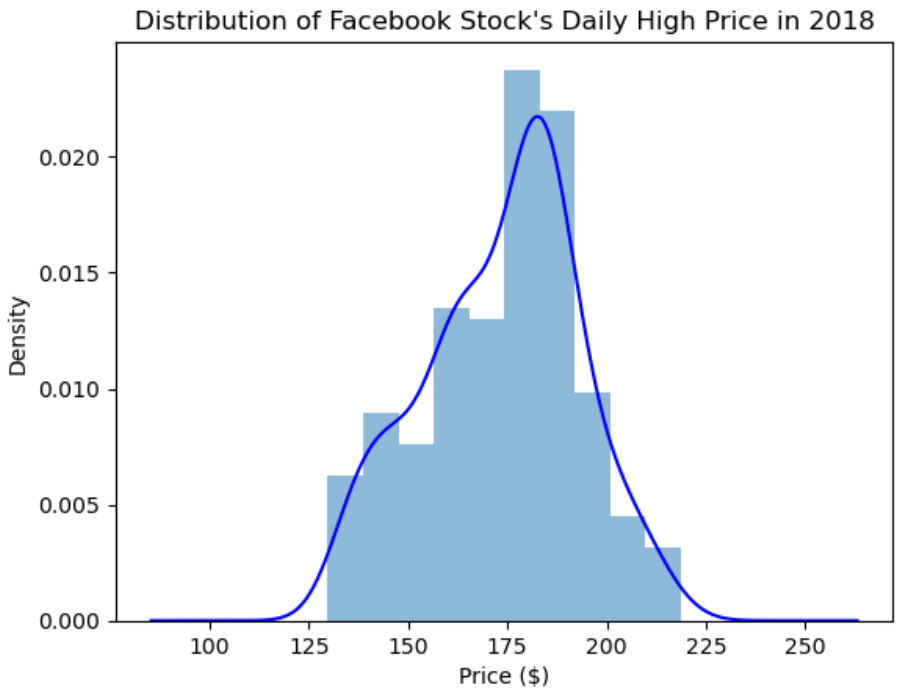
In [38]:
```
fb.high.plot(
    kind='kde',
    title='KDE of Daily High Price for Facebook Stock'
)
plt.xlabel('Price ($)') # label the x-axis (discussed in chapter 6)
```

Out[38]: Text(0.5, 0, 'Price ($)')



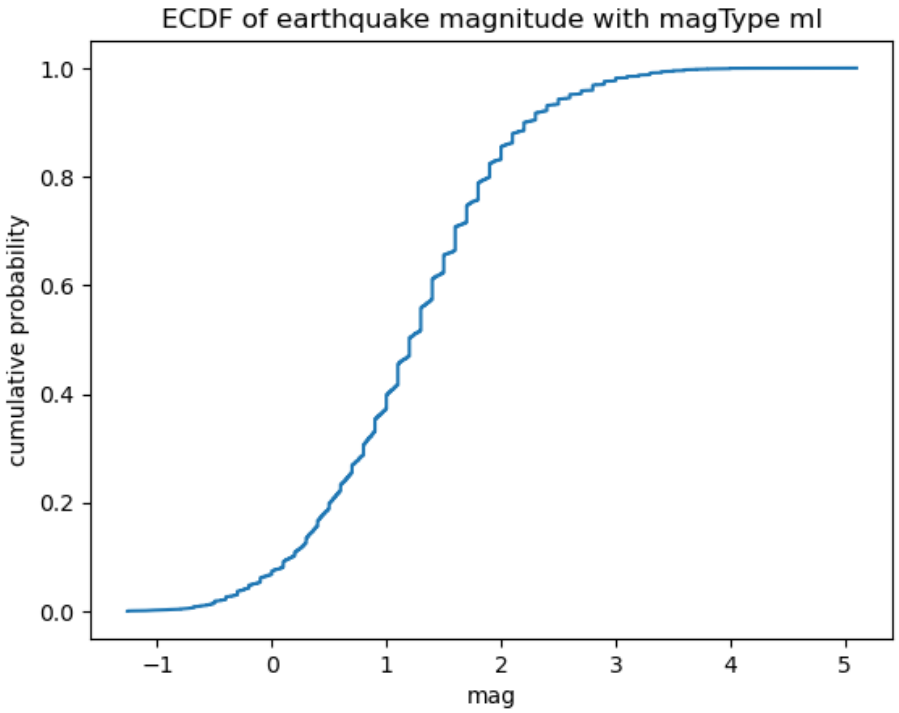
In [39]:
```
ax = fb.high.plot(kind='hist', density=True, alpha=0.5)
fb.high.plot(
    ax=ax, kind='kde', color='blue',
    title='Distribution of Facebook Stock\'s Daily High Price in
2018'
)
plt.xlabel('Price ($)') # label the x-axis (discussed in chapter 6)
```

Out[39]: Text(0.5, 0, 'Price ($)')



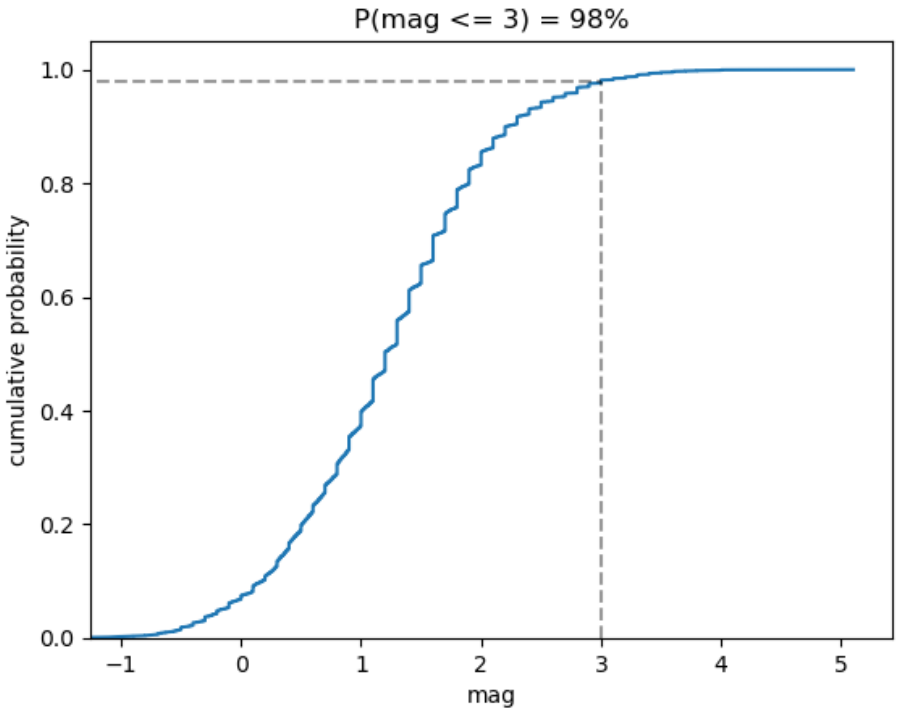
In [40]:
```
from statsmodels.distributions.empirical_distribution import ECDF

ecdf = ECDF(quakes.query('magType == "ml"').mag)
plt.plot(ecdf.x, ecdf.y)

# axis labels (we will cover this in chapter 6)
plt.xlabel('mag') # add x-axis label
plt.ylabel('cumulative probability') # add y-axis label

# add title (we will cover this in chapter 6)
plt.title('ECDF of earthquake magnitude with magType ml')
```

Out[40]: Text(0.5, 1.0, 'ECDF of earthquake magnitude with magType ml')



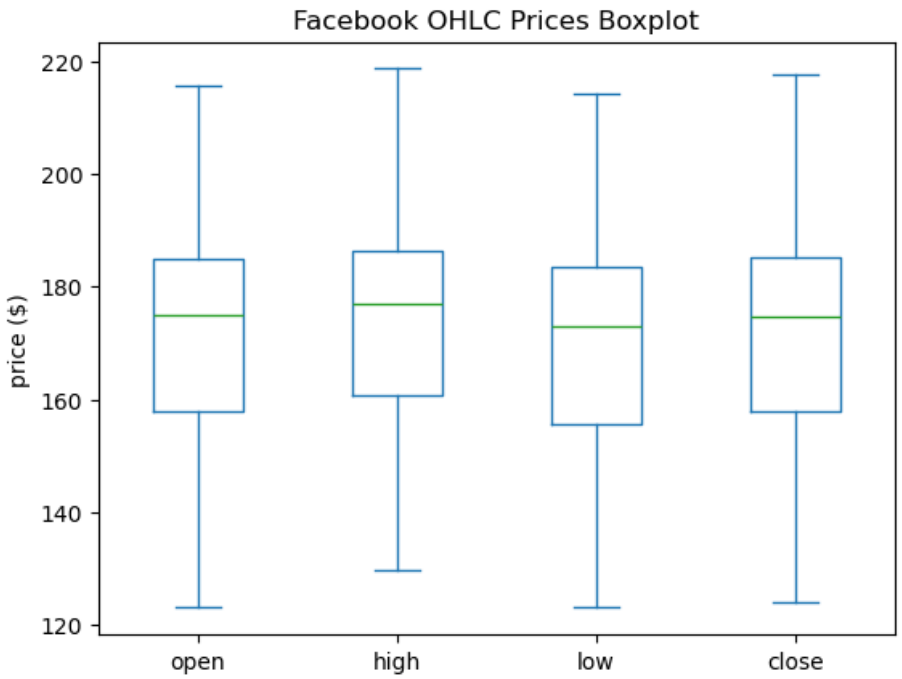
In [41]:
```
from statsmodels.distributions.empirical_distribution import ECDF

ecdf = ECDF(quakes.query('magType == "ml"').mag)
plt.plot(ecdf.x, ecdf.y)

# formatting below will all be covered in chapter 6
# axis labels
plt.xlabel('mag') # add x-axis label
plt.ylabel('cumulative probability') # add y-axis label

# add reference lines for interpreting the ECDF for mag <= 3
plt.plot(
    [3, 3], [0, .98], 'k--',
    [-1.5, 3], [0.98, 0.98], 'k--', alpha=0.4
)
# set axis ranges
plt.ylim(0, None)
plt.xlim(-1.25, None)

# add a title
plt.title('P(mag <= 3) = 98%')
```

Out[41]: Text(0.5, 1.0, 'P(mag <= 3) = 98%')



In [42]:
```
fb.iloc[:,:4].plot(kind='box', title='Facebook OHLC Prices Boxplot')
plt.ylabel('price ($)') # label the x-axis (discussed in chapter 6)
```

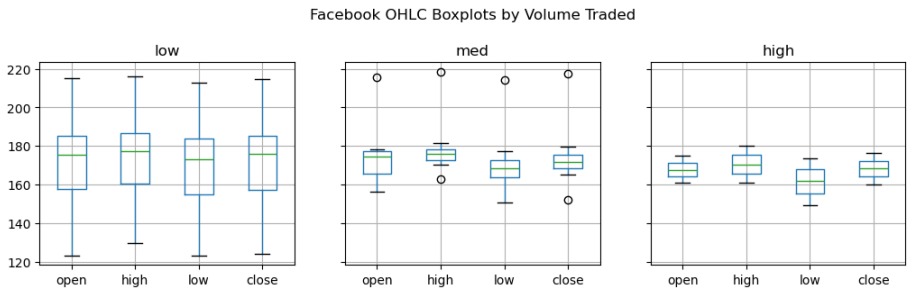
Out[42]: Text(0, 0.5, 'price ($)')



In [43]:
```
fb.assign(
    volume_bin=pd.cut(fb.volume, 3, labels=['low', 'med', 'high'])
).groupby('volume_bin').boxplot(
    column=['open', 'high', 'low', 'close'],
    layout=(1, 3), figsize=(12, 3)
)
plt.suptitle('Facebook OHLC Boxplots by Volume Traded', y=1.1)
```

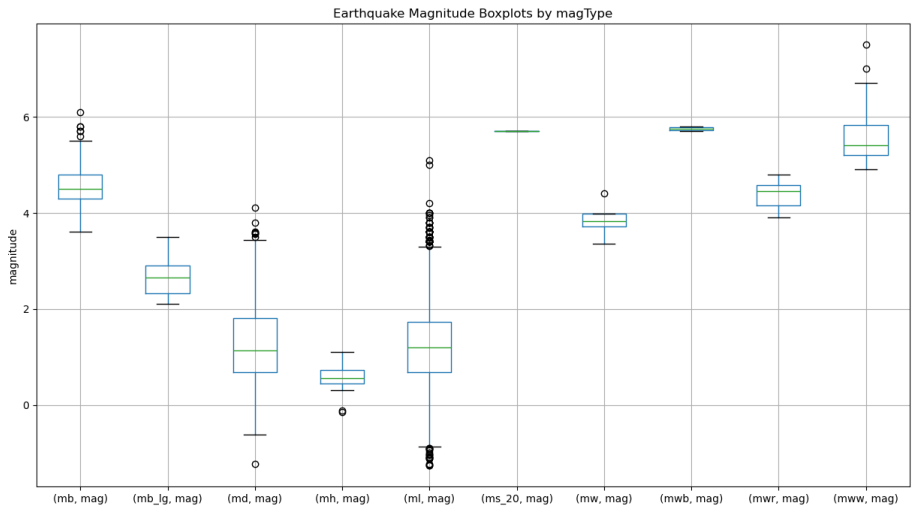
```
C:
\Users\tipqc\AppData\Local\Temp\ipykernel_8096\2255955200.py
:3: FutureWarning: The default of observed=False is
deprecated and will be changed to True in a future version
of pandas. Pass observed=False to retain current behavior or
observed=True to adopt the future default and silence this
warning.
  ).groupby('volume_bin').boxplot(
```

Out[43]: Text(0.5, 1.1, 'Facebook OHLC Boxplots by Volume Traded')



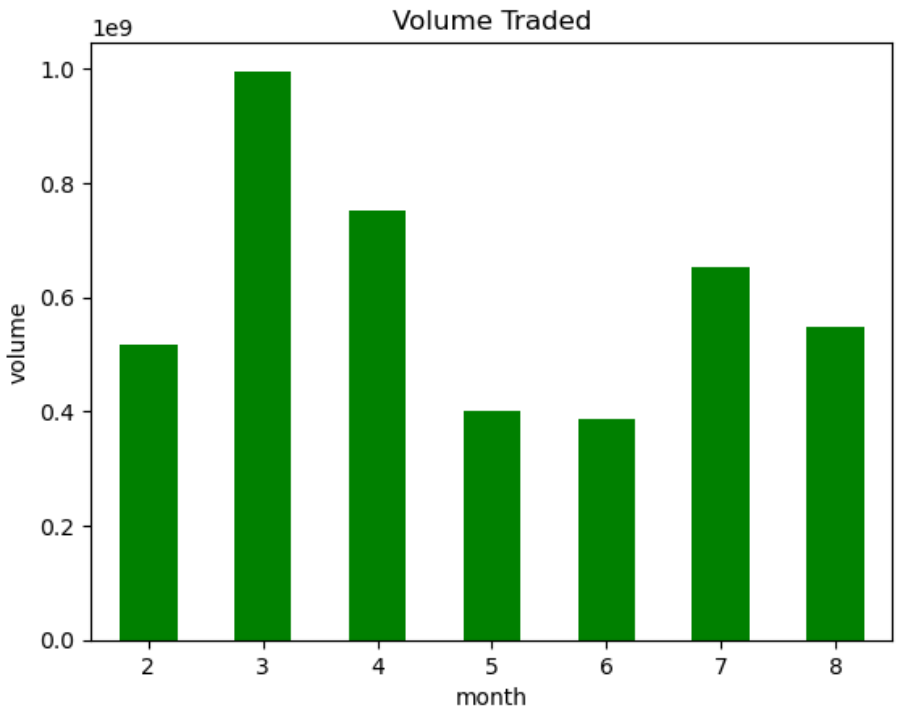
In [44]:
```
quakes[['mag', 'magType']].groupby('magType').boxplot(
    figsize=(15, 8), subplots=False
)
plt.title('Earthquake Magnitude Boxplots by magType')
plt.ylabel('magnitude') # label the y-axis (discussed in chapter 6)
```

Out[44]: Text(0, 0.5, 'magnitude')



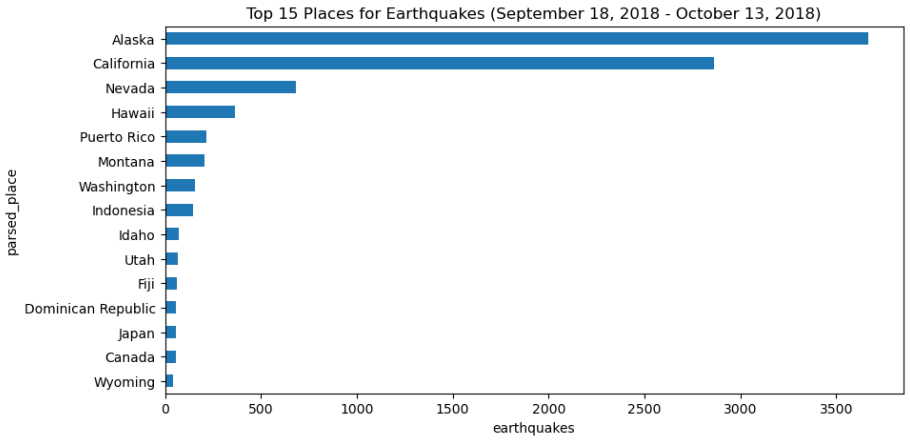
In [45]:
```
fb['2018-02':'2018-08'].assign(
    month=lambda x: x.index.month
).groupby('month').sum().volume.plot.bar(
    color='green', rot=0, title='Volume Traded'
)
plt.ylabel('volume') # label the y-axis (discussed in chapter 6)
```

Out[45]: Text(0, 0.5, 'volume')



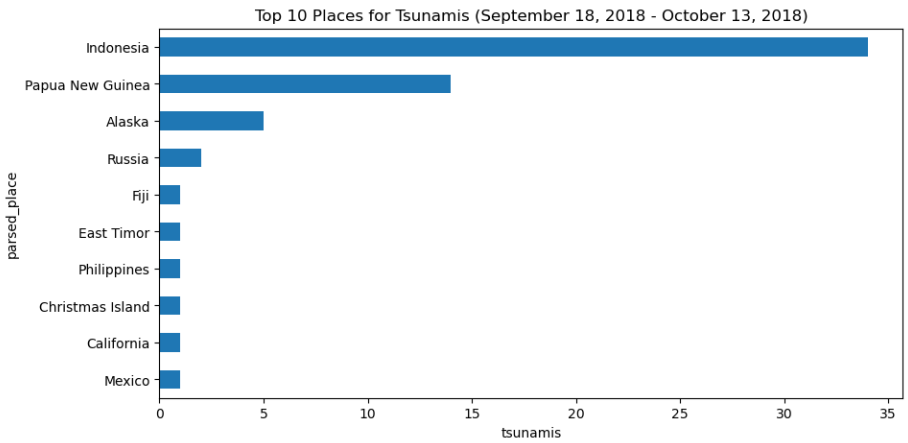
In [47]:
```
quakes.parsed_place.value_counts().iloc[14::-1,].plot(
    kind='barh', figsize=(10, 5),
    title='Top 15 Places for Earthquakes '\
        '(September 18, 2018 - October 13, 2018)'
)
plt.xlabel('earthquakes') # label the x-axis (discussed in chapter 6)
```

Out[47]: Text(0.5, 0, 'earthquakes')



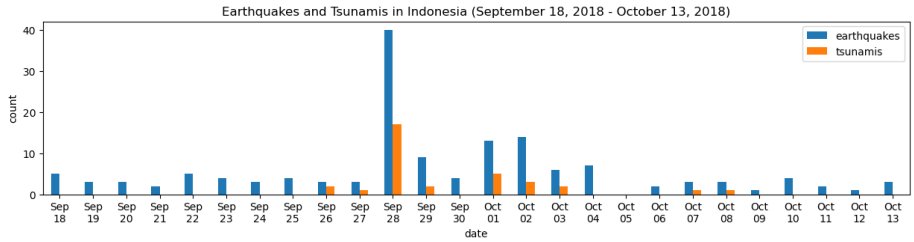
In [48]:
```
quakes.groupby('parsed_place').tsunami.sum().sort_values().iloc[-10::
,].plot(
    kind='barh', figsize=(10, 5),
    title='Top 10 Places for Tsunamis '\
        '(September 18, 2018 - October 13, 2018)'
)
plt.xlabel('tsunamis') # label the x-axis (discussed in chapter 6)
```

Out[48]: Text(0.5, 0, 'tsunamis')



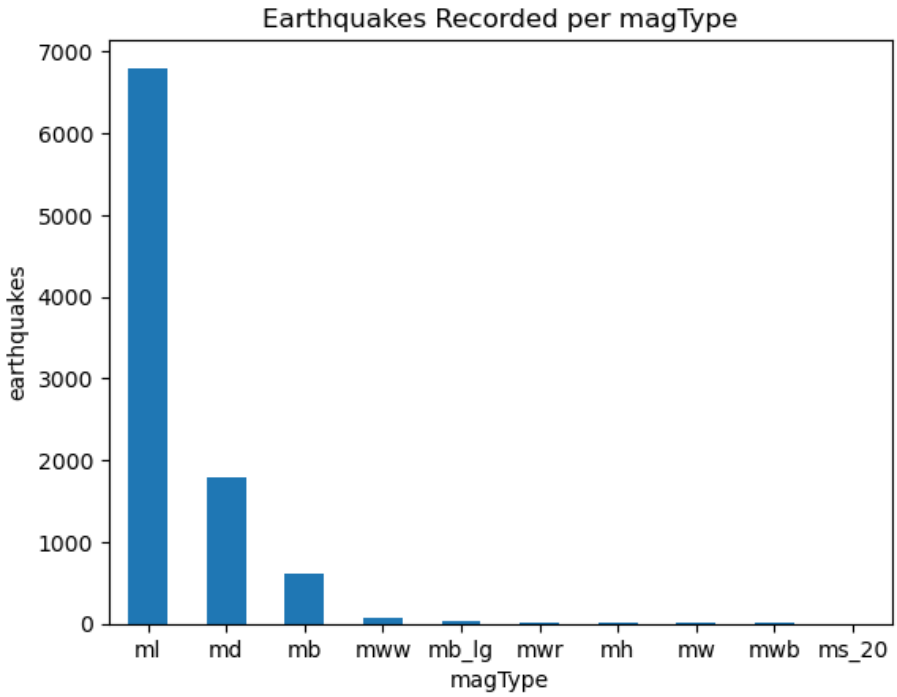
In [50]:
```
indonesia_quakes = quakes.query('parsed_place ==
"Indonesia"').assign(
    time=lambda x: pd.to_datetime(x.time, unit='ms'),
    earthquake=1
).set_index('time').resample('1D').sum()

indonesia_quakes.index = indonesia_quakes.index.strftime('%b\n%d')

indonesia_quakes.plot(
    y=['earthquake', 'tsunami'], kind='bar', figsize=(15, 3), rot=0,
    label=['earthquakes', 'tsunamis'],
    title='Earthquakes and Tsunamis in Indonesia '\
        '(September 18, 2018 - October 13, 2018)'
)
# label the axes (discussed in chapter 6)
plt.xlabel('date')
plt.ylabel('count')
```

Out[50]: Text(0, 0.5, 'count')



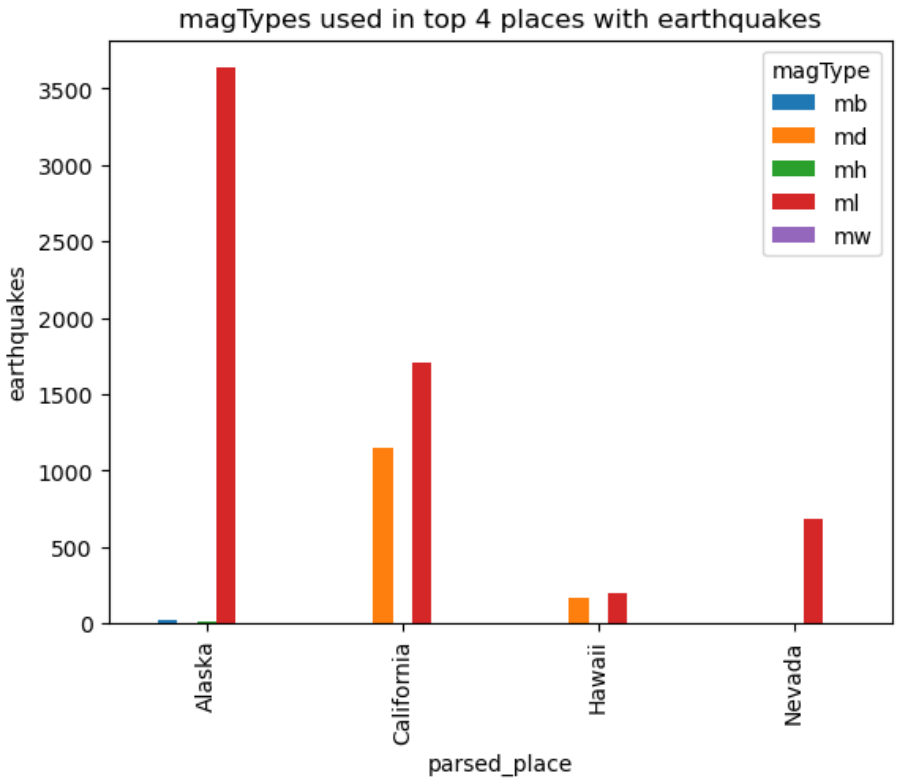
In [51]:
```
quakes.magType.value_counts().plot(
    kind='bar', title='Earthquakes Recorded per magType', rot=0
)
# label the axes (discussed in chapter 6)
plt.xlabel('magType')
plt.ylabel('earthquakes')
```

Out[51]: Text(0, 0.5, 'earthquakes')



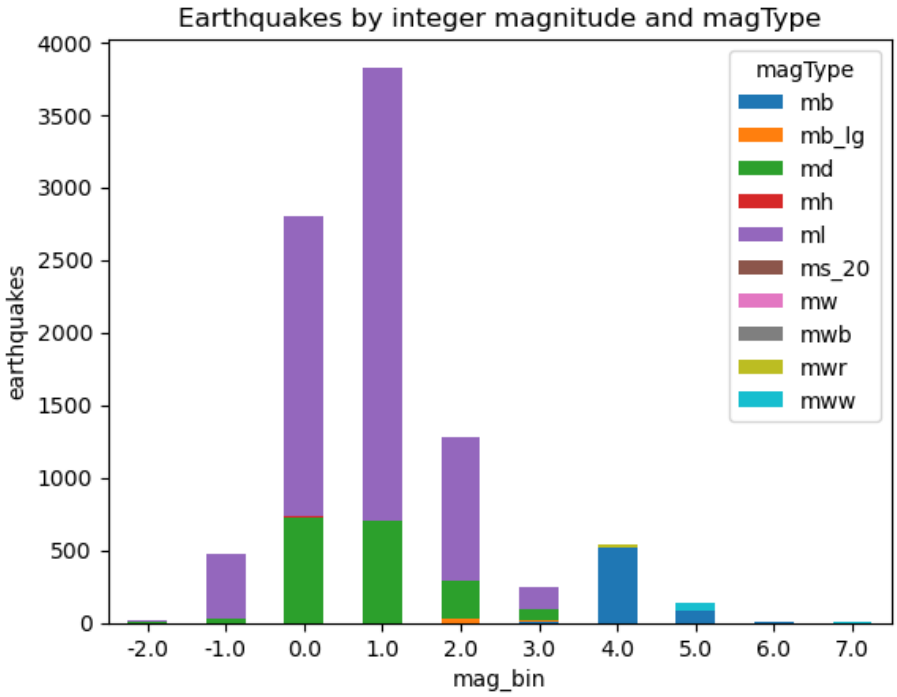
In [52]:
```
quakes[
    quakes.parsed_place.isin(['California', 'Alaska', 'Nevada',
'Hawaii'])
].groupby(['parsed_place',
'magType']).mag.count().unstack().plot.bar(
    title='magTypes used in top 4 places with earthquakes'
)
plt.ylabel('earthquakes') # label the axes (discussed in chapter 6)
```

Out[52]: Text(0, 0.5, 'earthquakes')



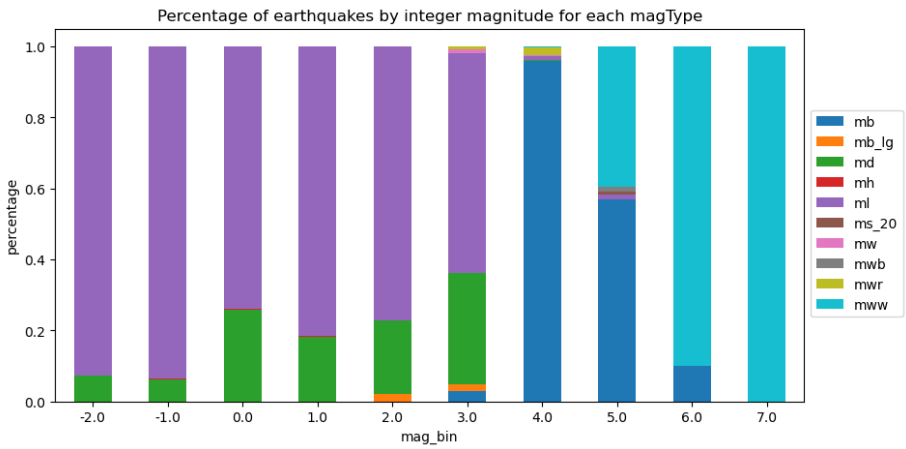
In [53]:
```
pivot = quakes.assign(
    mag_bin=lambda x: np.floor(x.mag)
).pivot_table(
index='mag_bin', columns='magType', values='mag', aggfunc='count'
)
pivot.plot.bar(
    stacked=True, rot=0,
    title='Earthquakes by integer magnitude and magType'
)
plt.ylabel('earthquakes') # label the axes (discussed in chapter 6)
```

Out[53]: Text(0, 0.5, 'earthquakes')



In [54]:
```
normalized_pivot = pivot.fillna(0).apply(lambda x: x/x.sum(), axis=1)
ax = normalized_pivot.plot.bar(
    stacked=True, rot=0, figsize=(10, 5),
    title='Percentage of earthquakes by integer magnitude for each
magType'
)
ax.legend(bbox_to_anchor=(1, 0.8)) # move legend to the right of the
plot
plt.ylabel('percentage') # label the axes (discussed in chapter 6)
```

Out[54]: Text(0, 0.5, 'percentage')

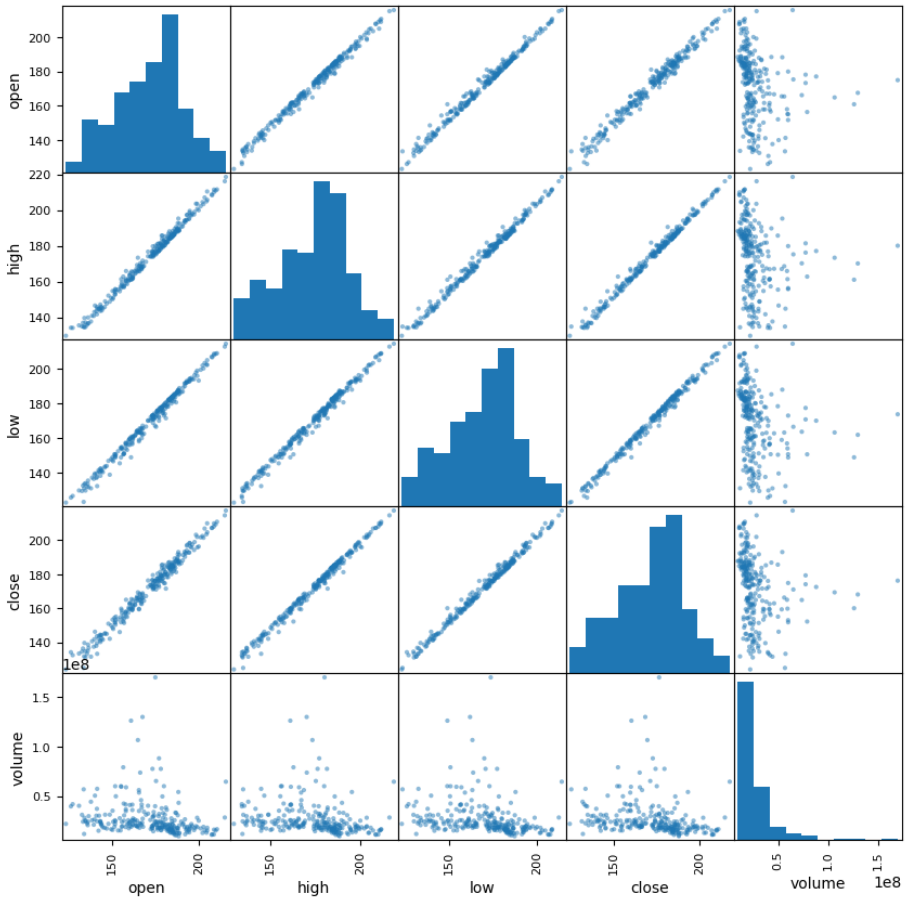


Pandas Plotting Subpackage

In [55]:
```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
fb = pd.read_csv(
'fb_stock_prices_2018.csv', index_col='date', parse_dates=True
)
```

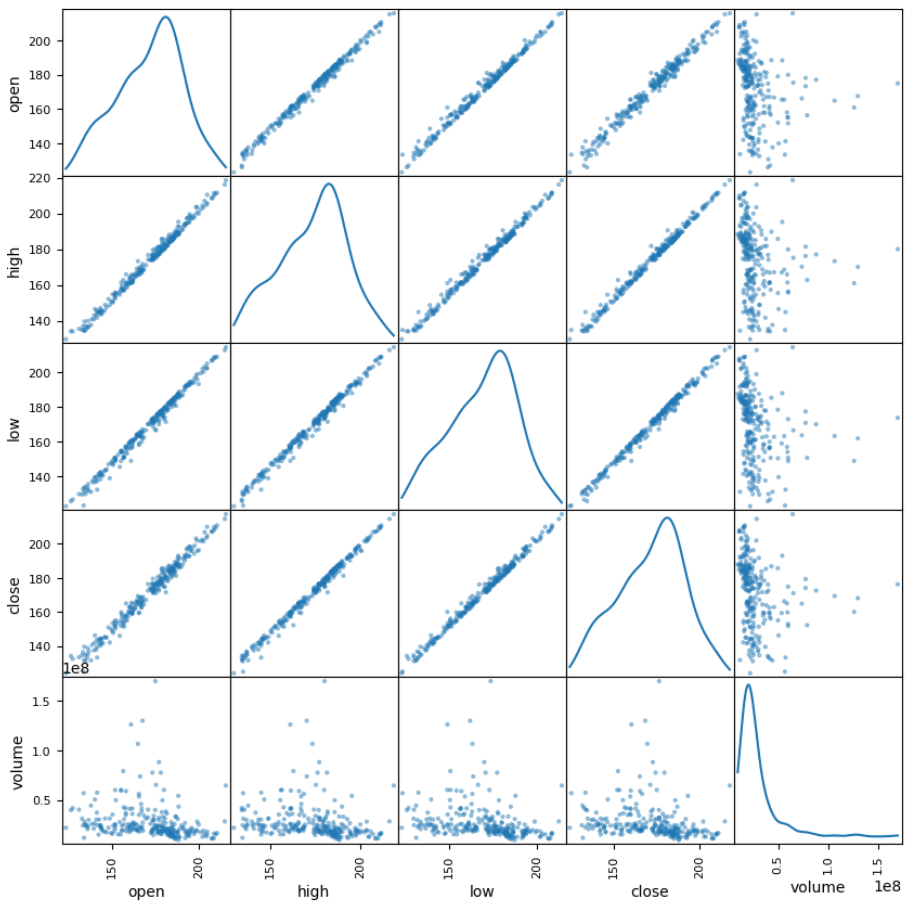
In [56]:
```
from pandas.plotting import scatter_matrix
scatter_matrix(fb, figsize=(10, 10))
```

Out[56]:
```
array([[<Axes: xlabel='open', ylabel='open'>,
        <Axes: xlabel='high', ylabel='open'>,
        <Axes: xlabel='low', ylabel='open'>,
        <Axes: xlabel='close', ylabel='open'>,
        <Axes: xlabel='volume', ylabel='open'>],
       [<Axes: xlabel='open', ylabel='high'>,
        <Axes: xlabel='high', ylabel='high'>,
        <Axes: xlabel='low', ylabel='high'>,
        <Axes: xlabel='close', ylabel='high'>,
        <Axes: xlabel='volume', ylabel='high'>],
       [<Axes: xlabel='open', ylabel='low'>,
        <Axes: xlabel='high', ylabel='low'>,
        <Axes: xlabel='low', ylabel='low'>,
        <Axes: xlabel='close', ylabel='low'>,
        <Axes: xlabel='volume', ylabel='low'>],
       [<Axes: xlabel='open', ylabel='close'>,
        <Axes: xlabel='high', ylabel='close'>,
        <Axes: xlabel='low', ylabel='close'>,
        <Axes: xlabel='close', ylabel='close'>,
        <Axes: xlabel='volume', ylabel='close'>],
       [<Axes: xlabel='open', ylabel='volume'>,
        <Axes: xlabel='high', ylabel='volume'>,
        <Axes: xlabel='low', ylabel='volume'>,
        <Axes: xlabel='close', ylabel='volume'>,
        <Axes: xlabel='volume', ylabel='volume'>]],
      dtype=object)
```


open
high
low
close
volume
1e8

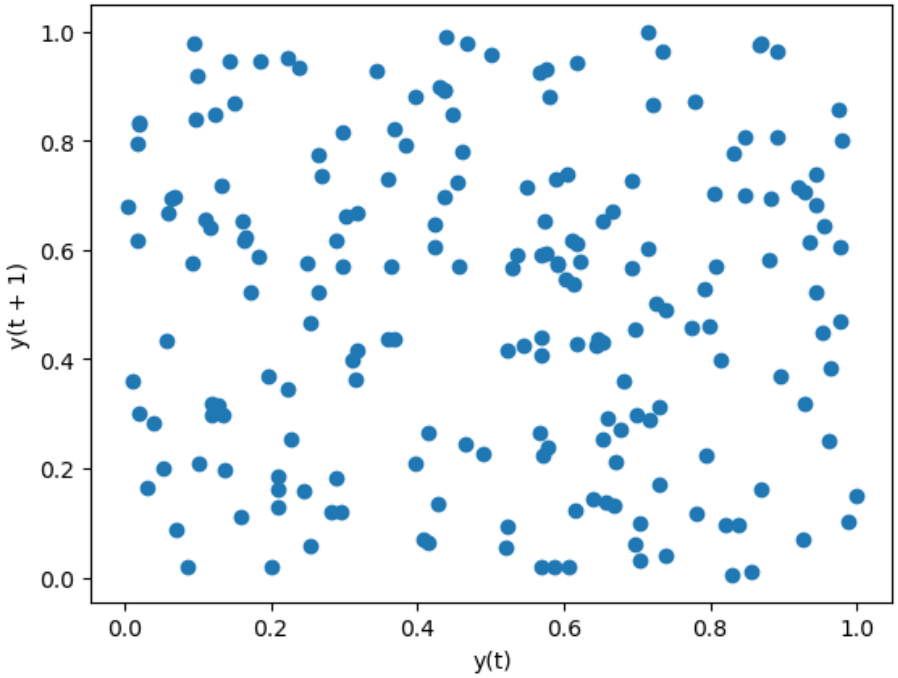
In [57]:
```
scatter_matrix(fb, figsize=(10, 10), diagonal='kde')
```

Out[57]:
```
array([[<Axes: xlabel='open', ylabel='open'>,
        <Axes: xlabel='high', ylabel='open'>,
        <Axes: xlabel='low', ylabel='open'>,
        <Axes: xlabel='close', ylabel='open'>,
        <Axes: xlabel='volume', ylabel='open'>],
       [<Axes: xlabel='open', ylabel='high'>,
        <Axes: xlabel='high', ylabel='high'>,
        <Axes: xlabel='low', ylabel='high'>,
        <Axes: xlabel='close', ylabel='high'>,
        <Axes: xlabel='volume', ylabel='high'>],
       [<Axes: xlabel='open', ylabel='low'>,
        <Axes: xlabel='high', ylabel='low'>,
        <Axes: xlabel='low', ylabel='low'>,
        <Axes: xlabel='close', ylabel='low'>,
        <Axes: xlabel='volume', ylabel='low'>],
       [<Axes: xlabel='open', ylabel='close'>,
        <Axes: xlabel='high', ylabel='close'>,
        <Axes: xlabel='low', ylabel='close'>,
        <Axes: xlabel='close', ylabel='close'>,
        <Axes: xlabel='volume', ylabel='close'>],
       [<Axes: xlabel='open', ylabel='volume'>,
        <Axes: xlabel='high', ylabel='volume'>,
        <Axes: xlabel='low', ylabel='volume'>,
        <Axes: xlabel='close', ylabel='volume'>,
        <Axes: xlabel='volume', ylabel='volume'>]],
dtype=object)
```



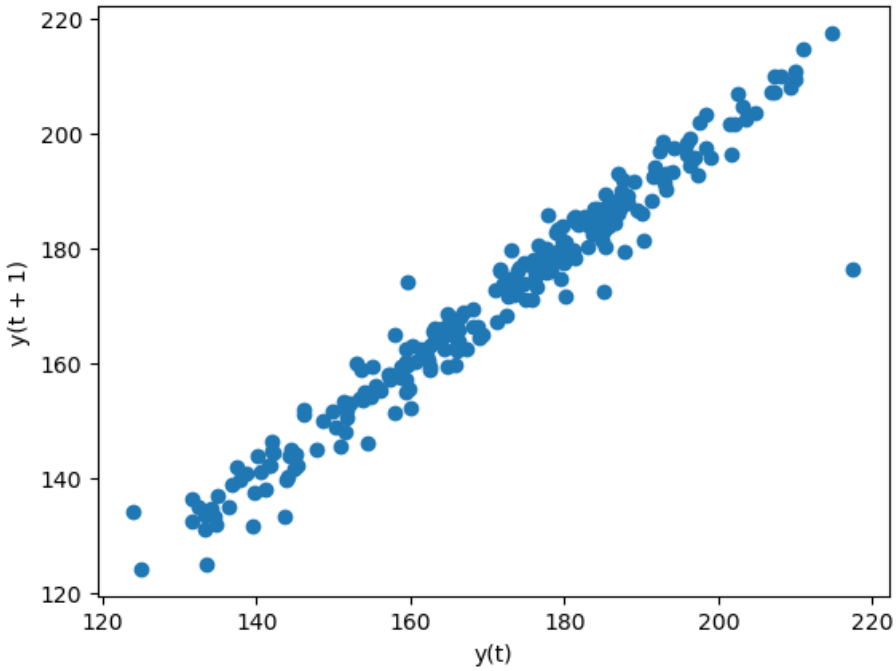
In [58]:
```
from pandas.plotting import lag_plot
np.random.seed(0) # make this repeatable
lag_plot(pd.Series(np.random.random(size=200)))
```

Out[58]: <Axes: xlabel='y(t)', ylabel='y(t + 1)'>



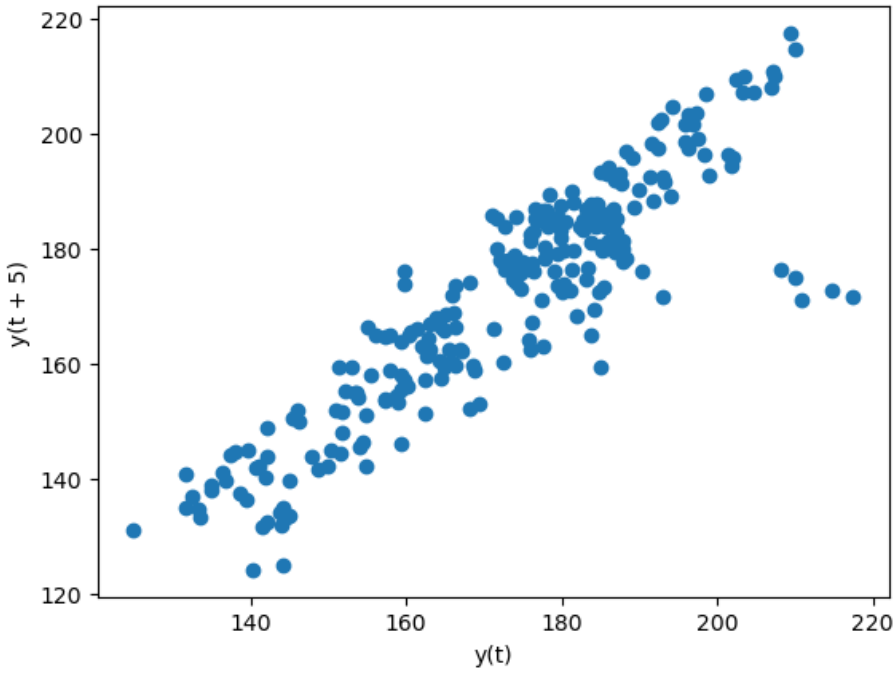
In [59]:
```
lag_plot(fb.close)
```

Out[59]: <Axes: xlabel='y(t)', ylabel='y(t + 1)'>



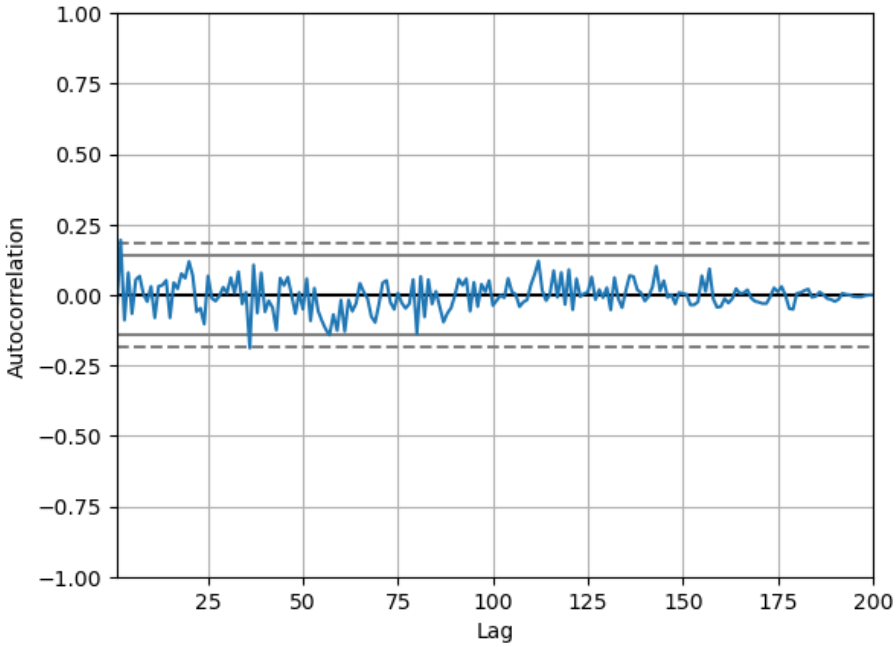
In [60]:
```
lag_plot(fb.close, lag=5)
```

Out[60]: <Axes: xlabel='y(t)', ylabel='y(t + 5)'>



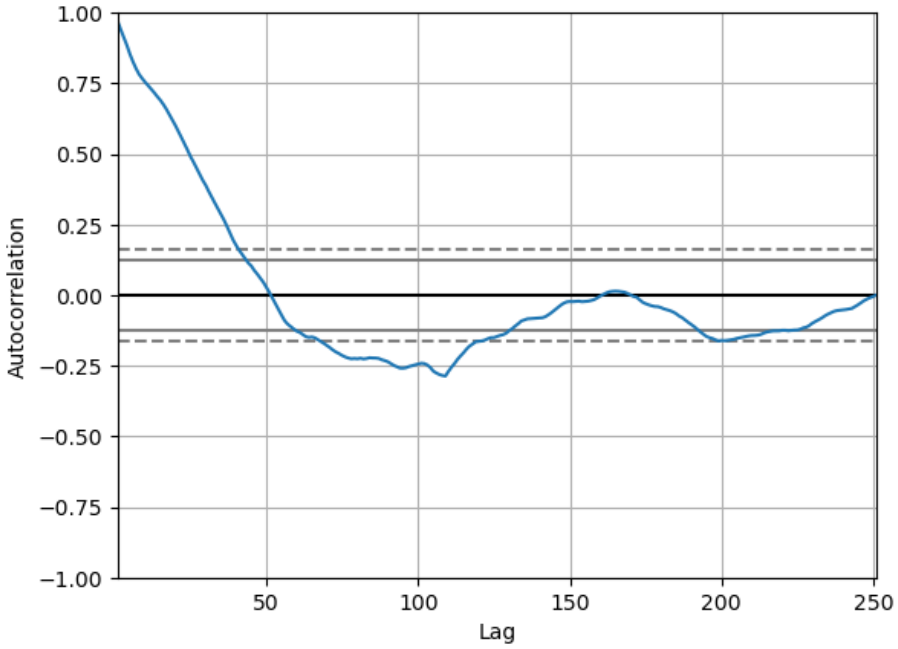
In [61]:
```
from pandas.plotting import autocorrelation_plot
np.random.seed(0) # make this repeatable
autocorrelation_plot(pd.Series(np.random.random(size=200)))
```

Out[61]: <Axes: xlabel='Lag', ylabel='Autocorrelation'>

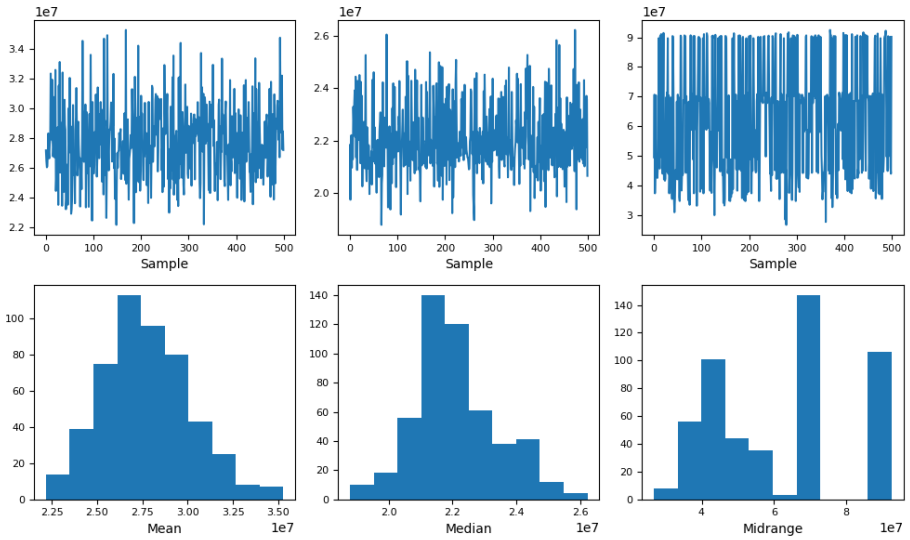


In [62]:
```
autocorrelation_plot(fb.close)
```

Out[62]: <Axes: xlabel='Lag', ylabel='Autocorrelation'>



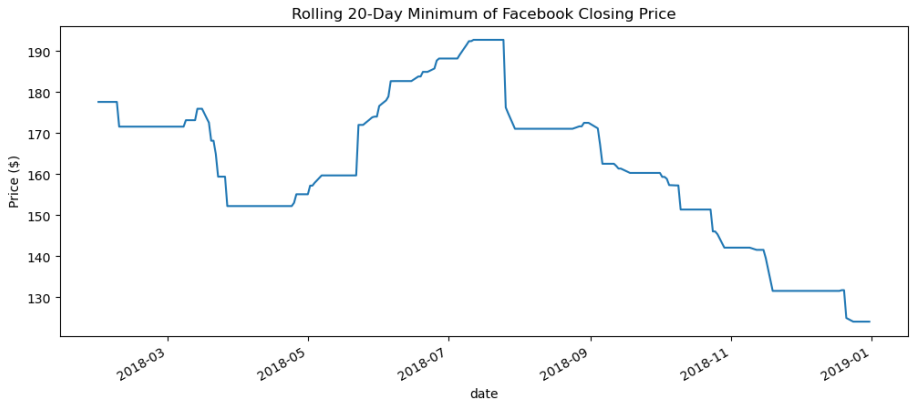
In [63]:
```
from pandas.plotting import bootstrap_plot
fig = bootstrap_plot(fb.volume, fig=plt.figure(figsize=(10, 6)))
```



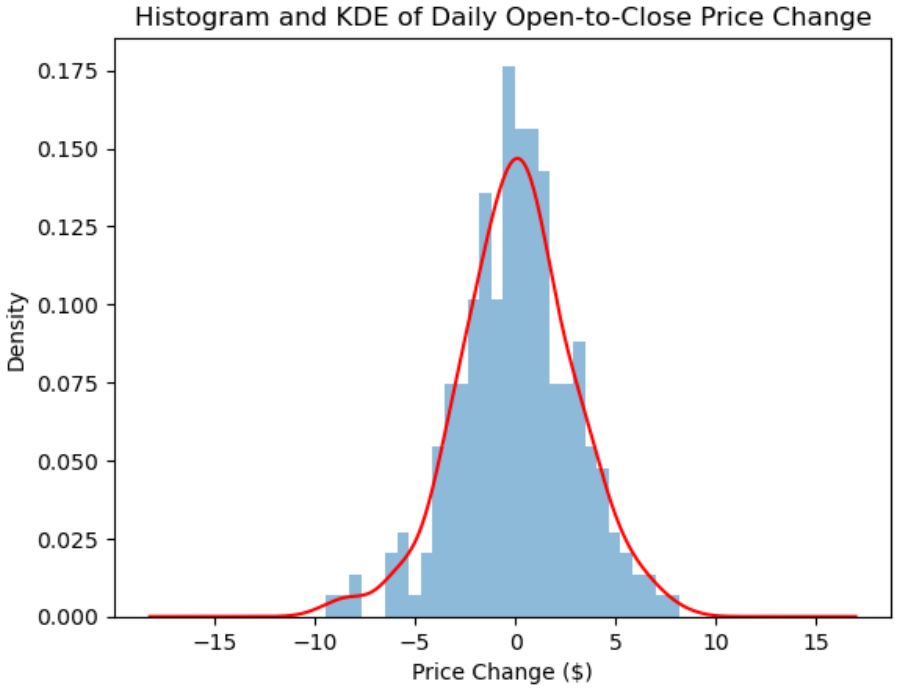
Supplementary Activity

In [3]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load the dataset
fb = pd.read_csv('fb_stock_prices_2018.csv', index_col='date',
parse_dates=True)
quakes = pd.read_csv('earthquakes.csv')
```

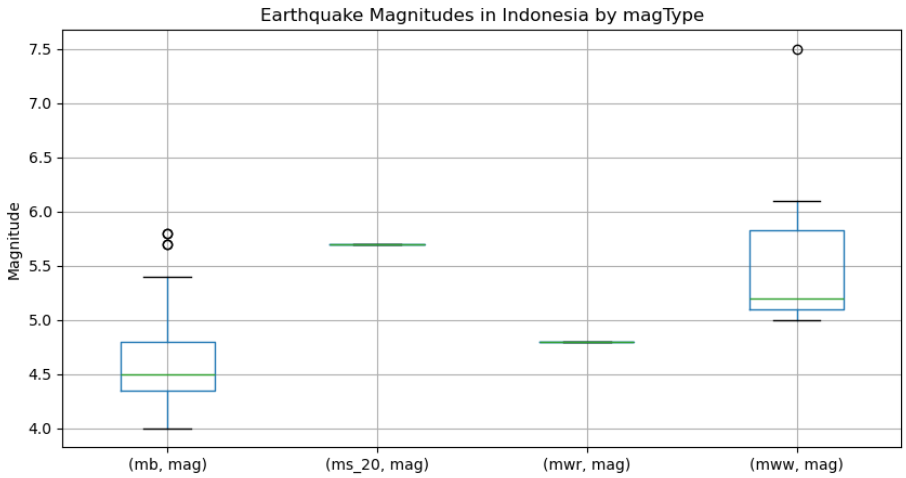
In [4]:
```
# 1. Rolling 20-day minimum of Facebook closing price
fb['close_20d_min'] = fb['close'].rolling(window=20).min()
fb['close_20d_min'].plot(
    title='Rolling 20-Day Minimum of Facebook Closing Price',
    figsize=(12, 5)
)
plt.ylabel('Price ($)')
plt.show()
```



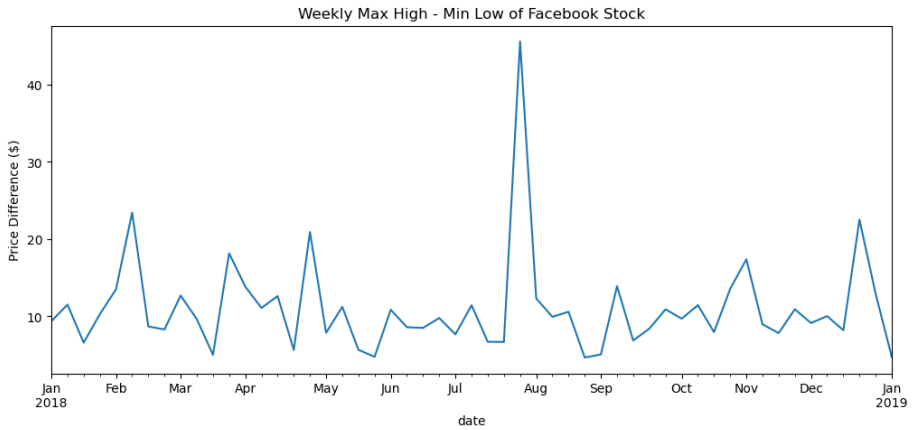
In [5]:
```
# 2. Histogram and KDE of the change from open to close
fb['price_change'] = fb['close'] - fb['open']
ax = fb['price_change'].plot.hist(
    bins=30, density=True, alpha=0.5,
    title='Histogram and KDE of Daily Open-to-Close Price Change'
)
fb['price_change'].plot.kde(ax=ax, color='red')
plt.xlabel('Price Change ($)')
plt.show()
```



In [6]:
```
# 3. Box plots of earthquake magnitudes for Indonesia by magType
indonesia_quakes = quakes.query('parsed_place == "Indonesia"')
indonesia_quakes[['mag', 'magType']].groupby('magType').boxplot(
    column='mag',
    figsize=(10, 5),
    subplots=False
)
plt.title('Earthquake Magnitudes in Indonesia by magType')
plt.ylabel('Magnitude')
plt.show()
```



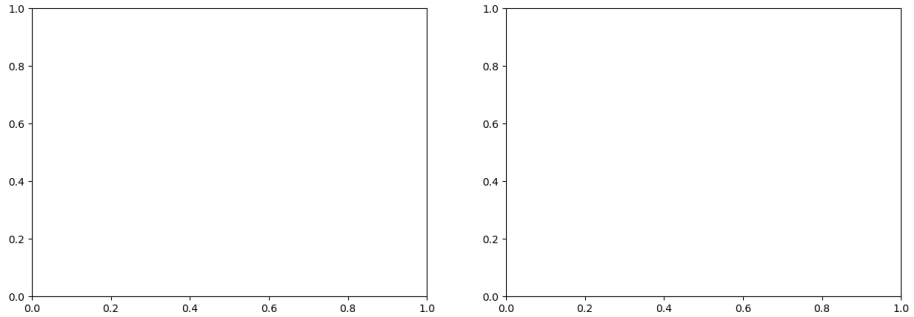
In [7]:
```
# 4. Weekly line plot: difference between weekly max high and weekly
min low
weekly_diff = fb['high'].resample('W').max() -
fb['low'].resample('W').min()
weekly_diff.plot(
    title='Weekly Max High - Min Low of Facebook Stock',
    figsize=(12, 5)
)
plt.ylabel('Price Difference ($)')
plt.show()
```



In [8]:
```
# 5. Subplots for after-hours trading effect
# a) Daily difference: today's open - previous day's close
fb['after_hours'] = fb['open'] - fb['close'].shift(1)
```

In [9]:
```
# b) Monthly net effect
monthly_effect = fb['after_hours'].resample('M').sum()

fig, axes = plt.subplots(1, 2, figsize=(15, 5))
```

```
C:
\Users\tipqc\AppData\Local\Temp\ipykernel_13804\2748946104.p
y:2: FutureWarning: 'M' is deprecated and will be removed in
a future version, please use 'ME' instead.
  monthly_effect = fb['after_hours'].resample('M').sum()
```



In [10]:
```
# Subplot 1: daily after-hours difference
fb['after_hours'].plot(
    ax=axes[0],
    title='Daily After-Hours Effect on Facebook Stock',
    color='blue'
)
axes[0].set_ylabel('Price Change ($)')
```

Out[10]: Text(4.444444444444452, 0.5, 'Price Change ($)')

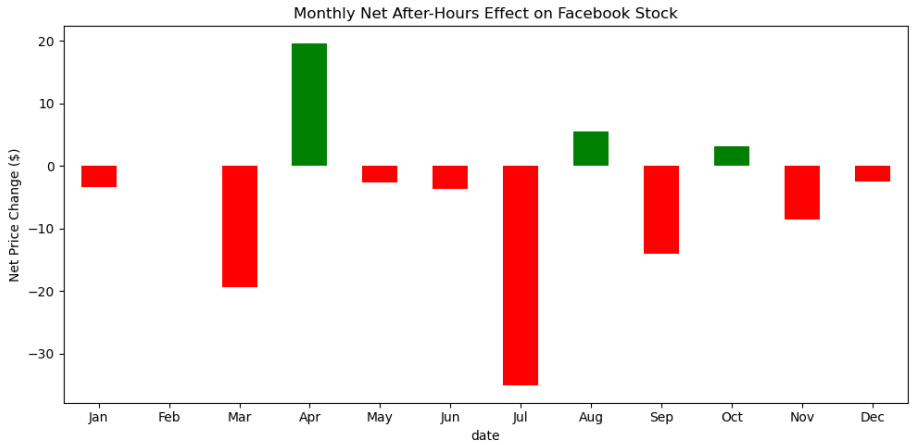
In [11]:
```
# Subplot 2: monthly net effect with colored bars
colors = ['green' if x >= 0 else 'red' for x in monthly_effect]
monthly_effect.plot(
    kind='bar',
    ax=axes[1],
    color=colors,
    title='Monthly Net After-Hours Effect',
    rot=0
)
axes[1].set_ylabel('Net Price Change ($)')
axes[1].set_xticklabels([x.strftime('%b') for x in
monthly_effect.index])  # Bonus #2

plt.tight_layout()
plt.show()
```

```
<Figure size 640x480 with 0 Axes>
```

In [12]:

```
import pandas as pd
import matplotlib.pyplot as plt

# Load dataset
fb = pd.read_csv('fb_stock_prices_2018.csv', index_col='date',
parse_dates=True)

# Compute daily after-hours effect: today's open - previous day's
close
fb['after_hours'] = fb['open'] - fb['close'].shift(1)

# Resample to monthly net effect
monthly_effect = fb['after_hours'].resample('M').sum()

# Determine bar colors based on gain or drop
colors = ['green' if val >= 0 else 'red' for val in monthly_effect]

# Plot
fig, ax = plt.subplots(figsize=(10, 5))
monthly_effect.plot(
    kind='bar',
    ax=ax,
    color=colors,
    title='Monthly Net After-Hours Effect on Facebook Stock'
)

# Set x-axis labels to 3-letter month abbreviations
ax.set_xticklabels([date.strftime('%b') for date in
monthly_effect.index], rotation=0)

# Label y-axis
ax.set_ylabel('Net Price Change ($)')

plt.tight_layout()
plt.show()
```

```
C:
\Users\tipqc\AppData\Local\Temp\ipykernel_13804\3348432799.p
y:11: FutureWarning: 'M' is deprecated and will be removed
in a future version, please use 'ME' instead.
  monthly_effect = fb['after_hours'].resample('M').sum()
```



Conclusion

The exercise boosted the strength of Pandas and MatplotLib in the analysis of financial and geophysical data. With the help of rolling statistics, resampling and visualization, trends, patterns, variability and anomalies in the stock prices and earthquake events can be determined. Colored bar charts and subplots improve the interpretability; they make complicated datasets more understandable and communicateable.

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.